

OBIX 1.0 — DOP-Native Component Language Specification

Draft 0.2

Field	Value
Document	OBIX 1.0 DOP-Native Component Language Specification
Draft	0.2
Date	28 August 2026
Supersedes	Draft 0.1 (<i>TDD Native Web Component Syntax</i>) where marked
Absorbs	<i>OBIX 1.0 Native Component Syntax Proposal</i> (28 August 2026) — the three-section design work
Author	OBINexus Computing — Nnamdi Michael Okpala
Status	Specification. Not implementation.

0. Preamble — what changed and why this draft exists

Draft 0.1 established the right *goals*: serializable state, pure transitions, deterministic render, accessibility as contract, native events, ES6 output, two test artifacts. Its authoring syntax — `component Timer { state {} action {} view {} }` — was a self-contained DSL that re-encoded HTML, CSS-adjacent structure, and a restricted expression language in one bespoke grammar.

Two forces move Draft 0.2 away from that:

1. The browser is the platform. A `view {}` block that re-spells HTML costs the language every HTML tool: validators, accessibility auditors, editor completion, the ARIA authoring practices corpus, and the ability to paste a working snippet from MDN. The three-section format (`<style>` / `<template>` / `<script>`) keeps CSS as CSS, HTML as HTML, and JavaScript as JavaScript, and adds OBIX semantics on top rather than in place of them.

2. OBIX is DOP-first, and DOP is the *middle* of the pipeline, not the end. The existing prototype already proved the important half of this:



Draft 0.2 generalises that result into the architecture:



The consequence for the language is stated once and enforced everywhere: **there is one component definition**. There is no `TimerFunctional.obix`, no `TimerOOP.obix`, no `TimerReactive.obix`. There is `Timer.obix`. The paradigm is a consumption decision made by the

importer, never an authoring decision made by the component. The whole point of the DOP adapter was to end the functional-versus-OOP duality; a language that made you pick at authoring time would reintroduce exactly the duality the adapter exists to dissolve.

The minimal invariant survives every layer:



```
state, actions, render }
```

The canonical IR is that invariant plus the things the compiler can prove:



```
name, state, actions, derived, validation, render, events, accessibility, metadata }
```

1. Decisions inherited unchanged from Draft 0.1

These are carried forward. Where Draft 0.2 refines them, the refinement is additive and the original decision stands.

#	Inherited decision	Status in 0.2
I1	<code>PascalCaseComponent.obix</code> is the component file	Unchanged. Refined: the filename is the <i>only</i> identity source (§2, superseded S1).
I2	<code>Component.test.obix</code> = behavioural / TDD specification	Unchanged. Refined syntax, same role.
I3	<code>Component.obix.test</code> = contract / conformance specification	Unchanged. Extended with adapter-equivalence and accessibility blocks.
I4	The two test artifacts are not aliases and have different audiences, lifecycles, and failure semantics	Unchanged. Draft 0.1's distinction is adopted verbatim as the rationale in §22–23.
I5	Contract validation runs first (fast, headless); behavioural tests second	Refined: <code>.test.obix</code> is <i>also</i> headless in 0.2; only accessibility-tree assertions need a DOM. Order preserved.
I6	State is plain, serializable data — no proxies, no observers, no hidden references	Unchanged and strengthened into a compile error (§8, <code>OBIX-J020</code>).
I7	Actions are pure transitions: state in, new state out	Unchanged and made canonical over the legacy mutable <code>ctx</code> model (§9, §29).
I8	Render is deterministic: same state → same output	Unchanged. Extended: one AST, two emitters, so SSR and client output cannot drift.
I9	Accessibility is contractual, not an afterthought	Unchanged and moved into compile time (§15).
I10	Events are web-native; no synthetic event system	Unchanged. Formalised as payload normalisation (§13).
I11	Output is ES6-native; no framework runtime, no virtual DOM, no production Node.js	Unchanged.
I12	Test-driven language design — tests are source artifacts, not external scripts	Unchanged.
I13	<code>given</code> / <code>when</code> / <code>expect</code> as the behavioural test triple	Restored. The Native Component Syntax Proposal used <code>dispatch</code> ; 0.2 returns to Draft 0.1's <code>when</code> (§22, C-note).
I14	Contract vocabulary: <code>contract</code> , <code>invariant</code> , <code>pre</code> , <code>post</code> , <code>old</code> , <code>unchanged</code> , <code>role</code> , <code>label</code> , <code>keyboard</code> , <code>focusable</code> , <code>aria</code>	Restored and specified. These keywords from Draft 0.1's lexical table become the <code>.obix.test</code> contract language (§23).
I15	Naming conventions: PascalCase components and actions, camelCase state members and helpers	Unchanged (§2.3).
I16	UTF-8 source, <code>//</code> and <code>/* */</code> comments in script	Unchanged.
I17	Optional test <code>layer</code> concept (state / render / integration)	Inherited in substance, dropped as a keyword. The layer is implied by the assertion subject and by which artifact it lives in (§22.4).
I18	<code>appsuite</code> , <code>app</code> , <code>domain</code> , <code>host</code> reserved for the application-suite layer	Reserved, undesigned. OBIXverse remains out of scope.

2. Decisions superseded from Draft 0.1

#	Draft 0.1 construct	Draft 0.2 replacement
S1	<code>component Timer { ... }</code> declaration	Filename <code>Timer.obix</code> is the sole identity
S2	<code>state { seconds: number = 0 }</code> block	<code><script>const state = { seconds: 0 };</code>

#	Draft 0.1 construct	Draft 0.2 replacement
S3	<code>action Start { running -> true }</code> with <code>-></code> transition operator	<code>actions.Start(state) → newState</code> returning a new object
S4	<code>when condition</code> action guard	<code>if (!condition) return state;</code> — the identity return
S5	<code>view { div id { attr = expr } }</code> block	<code><template></code> containing real HTML
S6	<code>text = expr</code> binding	<code>{path}</code> text interpolation
S7	<code>on click = Action</code>	<code>on:click="Action"</code>
S8	<code>on click = choose(running, Stop, Start)</code> — dynamic action selection in the view	One action that branches internally: <code>on:click="Toggle"</code>
S9	<code>label = "Reset"</code> as an element property	Element text content, or <code>aria-label</code> where the accessible name differs
S10	<code>choose(cond, a, b)</code> ternary in view and action expressions	<code>obix:if</code> in template; ordinary <code>?:</code> inside <code><script></code>
S11	Restricted expression language (<code>(+ - * / % == != and or not)</code>) with builtins <code>floorDiv</code> , <code>mod</code> , <code>padLeft</code>	ES6 expressions inside <code><script></code> ; no expression language in the template at all
S12	<code>function helperName(...)</code> block with declared types	Ordinary module-scope JavaScript functions, or <code>derived</code> members
S13	Keyword table containing <code>component</code> , <code>state</code> , <code>action</code> , <code>view</code> , <code>function</code> , <code>return</code> , <code>if</code> , <code>else</code> , <code>choose</code> , <code>typeof</code>	Deleted. OBIX reserves no JavaScript keywords because it does not parse JavaScript expressions.
S14	<code>.obix.lib</code> shared-library extension	Plain <code>.js</code> ES modules imported normally
S15	<code>layer state render integration</code> keyword	Implied by assertion subject and artifact (§22.4)
S16	Type annotations in state (<code>seconds: number = 0</code>)	Types inferred from the initial literal; declared explicitly in the <code>.obix.test surface {}</code> block where they are contractual
S17	Component-level <code>view</code> as the only render surface	<code><style></code> + <code><template></code> + <code><script></code> , three sections with distinct responsibilities
S18	Mutable <code>ctx.state.x = ...</code> transitions in the DOP prototype	Immutable transitions canonical; mutation demoted to a legacy compatibility adapter (§29)

3. Reasons for each breaking syntax change

Each change costs migration effort. Each therefore needs a reason that is not aesthetic.

3.1 `component Timer { ... }` → filename identity (S1)

Two sources of truth for one name is a defect class that can only exist if the second source exists. The filename is already load-bearing — imports resolve it, the module system uses it, the CSS scope token derives from it, DevTools shows it. Removing the declaration removes the possibility of `LoginForm.obix` containing `component Timer`. The braces also disappear, which un-indents the entire file by one level.

3.2 `state { seconds: number = 0 }` → `const state = { ... }` (S2, S16)

Draft 0.1's state block needs a parser for a typed-declaration mini-language whose initialiser grammar will, under pressure from real components, grow toward JavaScript anyway. A plain object literal is already: serializable by construction, checkable against JSON rules by the compiler, readable by every JS tool, and identical to what the DOP IR needs to emit. Types come from the literal for inference and from the contract's `surface {}` block where they must be *guaranteed* rather than merely known.

3.3 `action Start { running -> true }` → pure function (S3, S4)

The `->` operator can express only *per-property assignment from a restricted expression*. Real transitions need more: filtering an array, coupling two properties in one decision, early return, destructuring a payload. Draft 0.1 would have had to grow statements, and at that

point it is a programming language with a smaller standard library than the one already in the browser.

The `when` guard is worth a specific note because it is the one construct in 0.1 that was *better* than plain JS ergonomically:

```
function Tick { when running → Tick(state) {
  seconds -> seconds + 1      if (!state.running) return state;
  return { ...state, seconds: state.seconds + 1 };
}
```

The replacement is longer but strictly more capable, and the identity return `return state` is not merely equivalent — it is the **zero-work signal**. The runtime compares by `Object.is`, sees the same reference, and performs no update pass at all. This is the state-machine-minimisation principle from the OBIX tennis-tracker MVP surfacing in the render path: a transition that changes nothing is never recorded and never costs anything.

3.4 `view { ... }` → `<template>` (S5, S6, S7, S9)

This is the largest change and has the strongest reason. A `view` block is a dialect. Dialects lose:

Accessibility tooling. axe, Lighthouse, the ARIA authoring practices, screen-reader debugging, and the accessible-name computation algorithm all operate on HTML. A dialect must reimplement each of them or forfeit them. Since accessibility is a *contract* in OBIX (I9), forfeiting the tools is not survivable.

Copy-paste. Every correct HTML pattern on the web becomes a translation exercise.

Editor support. Attribute completion, tag closing, entity handling, formatting.

Ambiguity in the dialect itself. `label = "Reset"` — is that a `<label>`, the `label` attribute, the accessible name, or text content? In HTML there is no such question.

Draft 0.1's `div timerDisplay { role = "timer" }` is 0.2's `<div id="timerDisplay" role="timer">`, and the second one is checkable by tools that already exist.

3.5 `on click = choose(running, Stop, Start)` → `on:click="Toggle"` (S8)

Dynamic action selection puts a decision in the markup, and it breaks two things that matter more than the convenience:

The event→action map stops being static. The contract assertion `expect keyboard Enter dispatches Start` cannot be checked if the target action depends on runtime state.

The branch becomes untestable at the state layer. `when Toggle` with `given { running: true }` is a state-layer test; a `choose` in the view is only reachable through a render-layer test.

The replacement is one action containing the branch:

```
function Toggle(state) {
  return state.running ? actions.Stop(state) : actions.Start(state);
}
```

3.6 `floorDiv`, `mod`, `padLeft` → ES6 (S11, S12)

Draft 0.1 banned JavaScript expressions and therefore had to begin inventing a standard library. That library must be specified, documented, versioned, and — for a polyglot project — reimplemented in every target language. `Math.floor`, `%`, and `padStart` already exist, are already documented, and already behave identically in every browser.

The ban is retained where it earns its keep: **inside** `<template>`. Templates hold paths, not programs (§5.3). That restriction is what makes static accessibility analysis, dependency graphs, and SSR safety possible. Inside `<script>` there is no reason for it.

3.7 Mutable `ctx` → immutable transitions (S18)

```
function
```

```
// legacy prototype
toggle(ctx) { ctx.state.clicked = !ctx.state.clicked; }

// canonical 0.2
Toggle(state) { return {...state, clicked: !state.clicked; }; }
```

Mutation defeats: `Object.is` change detection (the previous state is gone, so nothing can be compared), undo and time-travel, telemetry replay, the halting/state-hinging cache model, and adapter equivalence testing (the Data adapter has no instance to mutate). It is not deprecated for style. It is deprecated because four separate features in the architecture depend on the previous state still existing after a transition. §29 defines the compatibility adapter so existing prototype code keeps running.

4. `<style>` semantics

4.1 Form

```
<html>
<style lang="css" type="scoped">
  .Timer { display: grid; gap: 1rem; }
</style>
```

`<style>` with no attributes is exactly `<style lang="css" type="scoped">`. **CSS is the default and canonical language; scoped is the default mode.**

4.2 `lang`

Value	Meaning
<code>css</code>	Default. Native CSS, passed through with scoping applied.
<code>scss</code>	Compiled by an externally registered build-time adapter <i>before</i> scoping.

Rule S10. `lang` selects a build-time adapter only. Preprocessor output must be plain CSS before scoping runs. `obixc` ships knowing `css` and nothing else; an unregistered `lang` is `OBIX-S010`. Preprocessors are never part of the browser runtime.

4.3 `type`

Value	Status	Meaning
<code>scoped</code>	default, shipping	Selectors match only elements owned by this component
<code>global</code>	shipping	Emitted unchanged into the application stylesheet
<code>module</code>	reserved, not implemented (<code>OBIX-S003</code>)	Hashed class names exposed as an imported map

`type` means *stylesheet mode*, not MIME type. `type="text/css"` is `OBIX-S002` with a message pointing at `lang`. `module` is reserved rather than shipped because attribute scoping already solves name collision, and implementing it would add a binding namespace and a directive to solve a solved problem.

4.4 Scoping mechanism

Every element originating in a component’s own template carries `data-obix="Timer"`. The attribute value is a **space-separated token list** and selectors are rewritten with `~=`:

Source	Output
<code>.Timer</code>	<code>.Timer[data-obix~="Timer"]</code>
<code>.Timer .Timer__button</code>	<code>.Timer .Timer__button[data-obix~="Timer"]</code>
<code>.Timer__button:hover</code>	<code>.Timer__button[data-obix~="Timer"]:hover</code>
<code>.Timer__display::after</code>	<code>.Timer__display[data-obix~="Timer"]::after</code>
<code>:global(.app-dark) .Timer</code>	<code>.app-dark .Timer[data-obix~="Timer"]</code>

Rule S32. A child component's root element carries both tokens: `[data-obix="ClockDisplay Timer"]`. The child owns its internals; the parent may position the child's root. The `[~=]` operator is what makes this work and removes any need for a deep-selector escape hatch.

Rule S33. Tokens are plain component names. Only when one build contains two same-named components from different paths do both receive a 4-character path-derived suffix (`Timer-a3f1`), with `OBIX-S030` naming both files. Nobody debugging a single-`Timer` project ever sees a hash. Shadow DOM is not used.

Rule S21. `@keyframes`, `@font-face`, `@property`, and `:root` hoist to global output with name prefixing (`Timer-spin`), since they have no element to scope to.

5. `<template>` semantics

5.1 Form

The body is HTML — parsed with HTML tokenisation rules for tags, attributes, comments, and text. Multiple roots are permitted. Void elements may be written `<img>` or `<img />`; non-void elements require closing tags.

5.2 The five recognised constructs

At every point the parser knows which of exactly five things it is looking at:

Construct	Written as	Recognised by
Literal HTML element	<code><section class="Timer"></code>	lowercase tag name
Component reference	<code><ClockDisplay /></code>	leading uppercase tag name
Interpolation	<code>{formattedTime}</code>	<code>{</code> in text or attribute value
Event binding	<code>on:click="Start"</code>	attribute name begins <code>on:</code>
Structural directive	<code>obix:if="running"</code>	attribute name begins <code>obix:</code>

Everything else is literal markup, copied through verbatim. Two prefixes only: `on:` **names something the browser already has** (an event type for `addEventListener`); `obix:` **names something only OBIX has**. Reserved `obix:` attributes, no others (`OBIX-T001`):

```

obix:if obix:else-if obix:else obix:for obix:key obix:slot

```

5.3 Interpolation and the path language

```

{path}

```

A single pair of braces around a **path**. No calls, no operators, no ternaries, no template literals.

```

html

```

```
{seconds}          <!-- ok -->
{formattedTime}    <!-- ok: derived -->
{state.seconds}    <!-- ok: qualified -->
{alarm.label}      <!-- ok: loop variable member -->
{validity.errors.email} <!-- ok: validity namespace -->

{seconds + 1}      <!-- OBIX-T020: expressions are not permitted -->
{format(seconds)}  <!-- OBIX-T021: declare a derived value -->
{running ? "on" : "off"} <!-- OBIX-T022: use obix:if or a derived value -->
```

Literal braces escape as `\{` and `\}`.

5.4 Binding namespaces and static resolution

A component has exactly **five** binding namespaces:

Namespace	Declared by	Qualified form
loop scope	<code>obix:for="alarm of alarms"</code>	<i>(unqualified only)</i>
props	<code>const props = { ... }</code>	<code>props.label</code>
derived	<code>const derived = { ... }</code>	<code>derived.formattedTime</code>
state	<code>const state = { ... }</code>	<code>state.seconds</code>
validity	<code>const validation = { ... }</code> (reserved, generated)	<code>validity.valid</code>

Rule T30. An unqualified head resolves at compile time in the order loop → props → derived → state. `validity` is always qualified and can never be shadowed (`OBIX-T033`).

Rule T31 — collisions are errors, not precedence. A head name present in more than one of props/derived/state makes the unqualified form `OBIX-T030`; the author must qualify. A loop variable shadowing any namespace is `OBIX-T031`. The resolution order therefore never arbitrates between two live candidates — every ambiguity is reported.

Rule T32. Unresolvable names are `OBIX-T032` with a did-you-mean. A typo never reaches the DOM as `undefined`.

So `{formattedTime}` means `derived.formattedTime(state, props)` — a name resolved to one declaration, checked at build time. Not `state.formattedTime`, not magic.

Rule T33. All interpolated text is written with `textContent` or `setAttribute`, never `innerHTML`. There is no raw-HTML interpolation directive. Injection through interpolation is structurally impossible.

5.5 Attributes

```
<div>
  <button type="button">                <!-- literal -->
  <div class="Timer__display Timer__display--{status}"> <!-- mixed -->
  <output aria-valuenow="{seconds}">      <!-- bound -->
  <button disabled="{isBusy}">            <!-- boolean -->
```

Rule T10 — boolean attributes. For HTML's boolean-attribute set (`disabled`, `checked`, `required`, `hidden`, `open`, `readonly`, `multiple`, `selected`, ...) a whole-value binding is presence/absence and never writes the string `"false"`.

Rule T11 — ARIA state attributes are string-valued. `aria-expanded`, `aria-invalid`, `aria-checked`, `aria-pressed`, `aria-busy`, `aria-hidden` require the literal strings `"true"`/`"false"` in the DOM:

```
<div>
  <button aria-expanded="{open}"> <!-- false → aria-expanded="false" -->
  <button disabled="{open}">      <!-- false → attribute removed -->
```

That asymmetry is a real accessibility bug source in hand-written HTML. The compiler resolving it by attribute name is one of the concrete arguments for the language existing.

Rule T12. A non-boolean attribute bound to `null` or `undefined` is removed; empty string renders `attr=""`.

Rule T13. Attribute *names* are always literal. `{key}="{value}"` is `OBIX-T012`. Prop spreading is not supported in 1.0 (§30, Q3).

5.6 Conditionals

```
html
<p obix:if="error">{error}</p>
<p obix:else-if="warning">{warning}</p>
<p obix:else>All good.</p>
```

Path-valued, evaluated for JavaScript truthiness. Chains must be adjacent siblings (`OBIX-T040`). `obix:else` takes no value (`OBIX-T041`). There is no `!` operator — declare the boolean you mean:

```
js
const derived = { hasAlarms: ({ alarms }) => alarms.length > 0 };
```

Compilation is an anchor comment plus a controller that creates or removes the branch subtree. `obix:if` on a component reference mounts/unmounts it, driving the `CREATED → UPDATED → HALTED → DESTROYED` lifecycle already defined by `ObixRuntime`.

5.7 Iteration

```
html
<li obix:for="alarm of alarms" obix:key="alarm.id">{alarm.label}</li>
<li obix:for="alarm, index of alarms" obix:key="alarm.id">{index} {alarm.label}</li>
```

The keyword is `of`, matching `for...of`; `in` is rejected (`OBIX-T050`) because `for...in` means something else in JavaScript.

Rule T51 — `obix:key` is mandatory (`OBIX-T051`). This is stricter than most frameworks and deliberate: an unkeyed list is the most common cause of lost focus and misapplied ARIA state on reorder, which is an accessibility defect, not a performance nit. Duplicate keys are a development-build error.

Rule T55. `obix:if` and `obix:for` on one element is `OBIX-T055`. Wrap one in the other rather than memorising a precedence rule.

Compilation is a keyed node map: nodes are created, moved with `insertBefore`, updated in place, or removed. Existing nodes are never rebuilt, so focus and transitions survive. This is a list-local algorithm over real DOM nodes — not a tree diff.

5.8 Composition and slots

```
html
<script>
import ClockDisplay from "./ClockDisplay.obix";
</script>
```

Rule T60. Explicit imports only. A PascalCase tag with no corresponding import is `OBIX-T060`. No global registry, no directory-scan auto-registration. The imported *binding* name is the tag name.

Props are declared in the child with defaults and validated at compile time when both files are in the compilation unit (`OBIX-T070` for an undeclared prop). Props flow down on parent update without recreating the child.

Slots exist because a `Card` or `Modal` without content insertion is unusable, and the alternative reintroduces `innerHTML`:

```
html
```

```
<!-- Card.obix -->
<div class="Card__body"><slot>No content.</slot></div>

<!-- parent -->
<Card title="Session">
  <h2 obix:slot="header">Timer session</h2>
  <p>Started at {startedAt}</p>
</Card>
```

Rule T80 — slot content is owned by the parent. It compiles in the parent's binding namespace, carries the parent's scope token, and dispatches parent actions. The child chooses *where*; the parent owns *what*. One rule, no ambiguity.

6. `<script>` semantics

6.1 Form

```
html
<script lang="js" type="component">
</script>
```

Attribute	Values	Meaning
<code>lang</code>	<code>js</code> (default), <code>ts</code>	Source dialect; TypeScript types are erased and never required
<code>type</code>	<code>component</code> (default), <code>test</code>	Compile mode; <code>test</code> is legal only in <code>.test.obix</code>

The body is a real ES module. It is handed to a standards-compliant ES parser and OBIX walks the resulting ESTree; **OBIX never forks JavaScript**. This is the reason Draft 0.1's DSL-inside-script alternative is not adopted: owning a JavaScript parser makes every future ECMAScript addition an OBIX maintenance event, and it breaks Prettier, ESLint, the TypeScript language service, and source maps on day one.

6.2 The six recognised bindings

The compiler recognises exactly six top-level `const` declarations by name:

Name	Required	Shape
<code>props</code>	no	plain object of defaults
<code>state</code>	yes if the template has state bindings	plain object literal
<code>actions</code>	no	object of pure functions
<code>derived</code>	no	object of pure functions
<code>validation</code>	no	object of rule declarations
<code>effects</code>	no	object of effect declarations

Everything else at top level is a module-private helper: usable by actions, derived values, and validation rules; invisible to the template; not exported.

Rule J1 — `export` is not written. The compiler synthesises the module's exports; an explicit `export` is `OBIX-J010` ("the component is the export"). **Rule J2** — the six bindings must be `const` (`OBIX-J011`). **Rule J3** — each must be a statically analysable literal at its declaration site (`OBIX-J012`); the compiler needs the key set at build time to construct the DOP IR.

6.3 Effects — the impure boundary

Actions are pure, so nothing above can make a clock tick. Effects fill that gap declaratively rather than by relaxing purity:



```
js
const effects = {
  tick: {
    every: 1000,
    while: ({ running }) => running,
    dispatch: "Tick"
  }
}
```

Key	Meaning	Compiles to
every: <ms>	recurring	<code>setInterval</code> / <code>clearInterval</code>
after: <ms>	one-shot	<code>setTimeout</code> / <code>clearTimeout</code>
on: "window resize"	global listener (<code>window</code> or <code>document</code>)	<code>addEventListener</code> / <code>removeEventListener</code>

`while` is a pure predicate over state, destructured for dependency declaration. The effect starts when it becomes true and stops when it becomes false. **The runtime owns every handle** — timer ids and listener references live in the instance, never in state. That is precisely what the serializable-state rule is protecting, and why `setInterval` inside an action is `OBIX-J032`.

Effects are not part of the reference semantics. They *cause* dispatches; they do not *perform* transitions. This matters for §20: adapter equivalence is defined over action sequences, and effects are simply one source of such a sequence. Async data (`fetch`) is deferred (§30, Q2).

7. The canonical DOP IR

7.1 What the IR is for

The IR is the artifact that the adapters consume. It exists so that:

- one component definition** can be presented as four paradigms without four sources;
- the **declaration half is pure JSON**, which makes it hashable, diffable, contract-checkable, transmissible, and readable by a non-JavaScript consumer (a Go, Rust, or C binding via the polyglot layer) without executing anything;
- contract drift is detectable** — `Component.obix.test` is checked against the IR, not against source text.

7.2 The split: declarations are data, implementations are code



```
timer.dop.json  ← declarations: pure JSON, no functions, no code
timer.js       ← function table: state initialiser, actions, derived, validation, render, mount
```

This split is the single most important structural decision in the IR. A JSON file containing `"fn": "actions.Tick"` is portable; a JSON file containing a function is not JSON. Every field that a tool, a contract runner, or a foreign-language binding needs to *reason about* lives in the JSON. Every field that must *execute* lives in the ES module and is referenced by name.

7.3 IR shape



```
TopComponent {
  name      : string
  irVersion  : "1.0"
  metadata   : Metadata
  state      : StateIR
  actions    : ActionIR[]
  derived    : DerivedIR[]
  validation : ValidationIR[]
  render     : RenderIR
  events     : EventIR[]
  accessibility : AccessibilityIR
}
```




```
propDeps    : string[] | ""

EventIR {
  id        : "e0"
  target    : NodePath
  type      : "click"
  modifiers : ("prevent"|"stop"|"once"|"capture")[]
  action    : "Start"
  payload   : PayloadIR[]           // normalisation spec (§13)
  delegated : boolean
}

PayloadIR =
  | { source: "event", accessor: "value"|"checked"|"key"|"files"|"detail"|"x"|"y"|"index" }
  | { source: "binding", ns, path }
  | { source: "literal", value }

AccessibilityIR {
  nodes      : A11yNodeIR[]
  tabOrder   : string[]
  headings   : { level, nodeId }[]
  liveRegions : { nodeId, politeness, atomic }[]
  idRefs     : { from, attribute, to, resolved }[]
  contracts  : A11yContractIR[]     // derived from the template, checkable
}
```

7.4 The minimal invariant

Every layer of the architecture, at every level of reduction, exposes:

```
☐ {state, actions, render }
```

`derived`, `validation`, `events`, `accessibility`, and `metadata` are **additive**. A consumer that understands only the minimal invariant can run any OBIX component correctly, just without derived caching, validity reporting, or contract checking. This is what makes the IR forward-compatible: adding a field to the IR can never break a minimal consumer.

7.5 IR invariants (checked by the compiler)

#	Invariant
IR1	<code>state.initial</code> is valid JSON — no functions, no DOM nodes, no class instances, no <code>Map</code> / <code>Set</code>
IR2	<code>state.keys</code> is closed; no action introduces a key outside it
IR3	Every <code>ActionIR.fn</code> , <code>DerivedIR.fn</code> , <code>ValidationIR.fn</code> resolves to a member of the emitted function table
IR4	Every <code>EventIR.action</code> names an existing <code>ActionIR</code> , with matching arity
IR5	Every <code>BindingIR.read</code> resolves to a namespace member
IR6	<code>DerivedIR</code> dependencies form a graph of depth 1 — no derived value reads another
IR7	The declaration half is JSON-round-trip stable: <code>parse(stringify(ir)) ≡ ir</code>
IR8	<code>metadata.hash</code> covers the declaration half only, so a comment change in <code><script></code> does not invalidate a contract

8. State semantics

```
js
const state = {
  seconds: 0,
  running: false,
  laps: []
};
```

Rule J10 — serializable. Permitted: string, number, boolean, `null`, array, plain object. Prohibited: functions, DOM nodes, `Map`/`Set`, class instances, promises, sockets, timers (`OBIX-J020`). This is inherited from Draft 0.1 (I6) and promoted from convention to compile error.

Three properties follow, and each one is load-bearing for something else in the architecture:

Serializable → SSR, telemetry, snapshot testing, time-travel, and the halting/state-hinging cache model that checkpoints a session and resumes it. None of these can checkpoint a socket handle.

Closed key set → the compiler builds a complete key → dependent-bindings map before the app runs, which is what makes shallow diffing a *complete* update strategy rather than an approximation.

Shallow-compared → after every transition the runtime compares keys with `Object.is`; identity return yields an empty changed set and the update pass exits immediately.

Rule J11. Non-deterministic initialisers (`Date.now()`, `Math.random()`) in the initial literal are `OBIX-J021` (warning): the value differs between server render and client hydrate. Pass it as a prop or set it from an effect.

Rule J12. Actions may not add keys absent from the initial literal (`OBIX-J022`) — statically where the returned literal is analysable, at runtime in development builds where it is not.

Rule J13. Nested objects and arrays are permitted and compared by identity at the top level only. An action returning a new top-level object for a changed nested value updates correctly; an action mutating a nested object in place does not — and Rule J21 makes that mutation an error anyway. Stated plainly rather than hidden (§30, Q6).

9. Action semantics

```
js
const actions = {
  Start(state) {
    if (state.running) return state;
    return { ...state, running: true };
  },
  SetLabel(state, label) {
    return { ...state, label };
  },
  RemoveLap(state, id) {
    return { ...state, laps: state.laps.filter((lap) => lap.id !== id) };
  }
};
```

Rule J20 — signature. `ActionName(state, ...payload) → newState`. First parameter is always current state. Payload comes from event normalisation (§13) or from an explicit `dispatch` call.

Rule J22 — action names are PascalCase (`OBIX-J036`), inherited from Draft 0.1 (I15). This makes `on:click="Start"` visually a *dispatch* rather than a function call, and separates actions from camelCase module-private helpers at a glance.

Rule J21 — purity is enforced by lint, honestly. JavaScript purity cannot be proven, so the compiler enforces a checkable subset and says so:

Check	Code	Level
Assignment to the state parameter or its members	<code>OBIX-J030</code>	error
<code>document</code> , <code>window</code> , <code>navigator</code> , <code>localStorage</code> , <code>fetch</code> , <code>XMLHttpRequest</code>	<code>OBIX-J031</code>	error
<code>setTimeout</code> / <code>setInterval</code> / <code>requestAnimationFrame</code>	<code>OBIX-J032</code>	error — Use <code>effects</code>
<code>async</code> action or returned promise	<code>OBIX-J033</code>	error
Reference to a top-level <code>let</code> / <code>var</code>	<code>OBIX-J034</code>	warning — hidden mutable state
Missing return on some path	<code>OBIX-J035</code>	error

These are diagnostics, not a sandbox. The compiler’s job is to make the correct thing easy and the incorrect thing loud.

Rule J23 — the identity return is the zero-work signal. `return state` unchanged produces no observable transition in any adapter (§20, E4). It is the direct successor of Draft 0.1’s `when` guard and the render-path expression of state-machine minimisation.

Rule J24 — actions may call other actions. `Toggle(state) { return state.running ? actions.Stop(state) : actions.Start(state); }` is legal and composes purely. Recursion is `OBIX-J037`.

10. Derived-value semantics

```
js
const derived = {
  formattedTime({ seconds }) {
    const minutes = Math.floor(seconds / 60);
    const remainder = seconds % 60;
    return `${String(minutes).padStart(2, "0")}:${String(remainder).padStart(2, "0")}`;
  },

  statusLabel({ running, seconds }, { idleText }) {
    if (running) return "Running";
    if (seconds > 0) return "Paused";
    return idleText;
  }
};
```

Rule J40 — signature. `name(state, props) → value`. Pure, no side effects, same lint set as actions.

Rule J41 — dependencies are declared by destructuring. This is the mechanism that gives fine-grained updates without proxies, observers, or runtime tracking:

```
js
formattedTime({ seconds })           // depends on state.seconds
statusLabel({ running }, { idleText }) // state.running, props.idleText
elapsedRatio(state)                  // parameter not destructured → depends on ALL state
```

A destructured parameter names its dependencies *syntactically*, in ordinary ES6 a developer would plausibly have written anyway. A non-destructured parameter is conservative and correct — recomputed on any state change — and produces `OBIX-J041` (advisory). Rest patterns fall back to the conservative set.

Rule J42 — derived values may not reference other derived values (`OBIX-J042`). Shared work goes in a module-private helper called from both. This keeps the dependency graph one level deep and acyclic by construction (IR6).

Rule J43 — evaluation timing. Evaluated when a binding needs a value: once at mount, thereafter only when the declared dependency set intersects the changed-key set. Memoised per update pass, so a derived value used by six bindings computes once.

Rule J44 — derived values are never stored in state. They are a projection, not a fact.

11. Validation semantics

Draft 0.1 listed `contract`, `pre`, `post`, and `invariant` as keywords but did not give the component a validation surface. Draft 0.2 adds one, because the existing 30-component library is form-heavy: `error`, `valid`, `ariaInvalid`, `ariaDescribedBy`, and `validation: "blur" | "change" | "submit"` already appear across the runtime, wired by hand in every component. Validation belongs in the IR so it can be wired once and asserted contractually.

11.1 Declaration

```
js
const validation = {
  emailFormat: {
    field: "email",
    severity: "error",
    message: "Enter a valid email address.",
    rule: ({ email }) => email === "" || /^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(email)
  },

  durationLimit: {
    field: "seconds",
    severity: "warning",
    message: "Approaching the maximum duration.",
    rule: ({ seconds }, { limitSeconds }) => seconds < limitSeconds * 0.9
  }
}
```

Key	Required	Meaning
<code>rule</code>	yes	Pure predicate <code>(state, props) → boolean</code> ; true means valid
<code>message</code>	yes	String shown when the rule fails
<code>severity</code>	no	<code>"error"</code> (default) or <code>"warning"</code>
<code>field</code>	no	State key this rule is <i>about</i> , used for accessibility wiring

Dependencies are declared by destructuring, exactly as for derived values (J41).

11.2 The `validity` namespace

After every transition, before render, the runtime evaluates every rule whose dependencies changed and produces a reserved, read-only namespace:

```
js
validity = {
  valid: boolean,           // no error-severity rule failing
  errors: { emailFormat: string|null },
  warnings: { durationLimit: string|null },
  invalid: { email: boolean }, // keyed by `field`
  messages: { email: string|null } // first failing message per field
}
```

Templates read it like any other namespace:

11.3 Rules

Rule V1 — validation is descriptive, never prohibitive. A failing rule does not block or roll back a transition. State remains the single source of truth about what happened; validity is a projection describing whether what happened is acceptable. Blocking transitions would make actions non-deterministic from the caller's perspective and would break adapter equivalence (§20), because a blocking rule turns `reduce` into a partial function.

Guard behaviour belongs in the action, where it is visible and testable:

```
js
Submit(state) {
  return state.emailValid ? { ...state, submitted: true } : state; // identity return
}
```

Rule V2 — `validity` is not state. It is not serialized, not diffed, not stored; it is recomputed from state. It cannot be assigned, and it cannot be shadowed (`OBIX-T033`).

Rule V3 — rules are pure and carry the same lint set as actions and derived values.

Rule V4 — accessibility wiring is checked. If a rule declares `field: "email"`, the compiler requires that a form control bound to `email` exists and that its `aria-invalid` is bound to `validity.invalid.email`, or emits `OBIX-A014` (warning). If an error message node reads `validity.messages.email`, it must be reachable from the control via `aria-describedby`, or `OBIX-A008`.

Rule V5 — validation declarations generate contract invariants. Every rule appears in the IR, so `Component.obix.test` can assert it without restating it:

```
invariant EmailFormat {
  given state { email: "not-an-email" }
  expect validity.invalid.email == true
  expect aria-invalid on EmailInput == "true"
```

This closes the loop that Draft 0.1's FUD policy layer was reaching for: the uncertainty-and-doubt half of FUD is precisely "did my input get accepted, and if not, why" — and that is now a declared, compiled, contractually asserted part of the component rather than per-component hand wiring.

Rule V6 — validation timing modes. The existing runtime's `validation: "blur" | "change" | "submit"` is a *presentation* choice about when to show a message, not about when to compute it. Rules always evaluate; a component chooses display timing with ordinary state:

```
js
const state = { email: "", touched: false };
```

```
html
```

```
<p obix:if="showEmailError">{validity.messages.email}</p>
```

```
js
const derived = { showEmailError: ({ touched }) => touched };
```

12. Native event mapping

```
on:<eventtype>[.modifier]* = "<ActionName>[(<arg>, ...)]"
```

```
html
<button on:click="Start">Start</button>
<input on:input="SetEmail(event.value)">
<form on:submit.prevent="Submit">
<button on:click="RemoveLap(lap.id)">Delete</button>
```

Rule E0 — direct mapping. `on:click` compiles to `addEventListener("click", ...)`. No synthetic event system, no event pooling, no proprietary bus (I10). `CustomEvent` from a child bubbles like any other event and is bound the same way.

Rule E4 — the action must exist with matching arity (`OBIX-T092` / `OBIX-T093`).

Modifiers — exactly four, each mapping 1:1 onto something native:

Modifier	Compiles to
<code>.prevent</code>	<code>event.preventDefault()</code> before dispatch
<code>.stop</code>	<code>event.stopPropagation()</code> before dispatch
<code>.once</code>	<code>addEventListener(..., { once: true })</code>
<code>.capture</code>	<code>addEventListener(..., { capture: true })</code>

`.prevent` and `.stop` exist because they are DOM method calls that would otherwise force impurity into an action. `.once` and `.capture` are literally `addEventListener` options. There is no `.debounce`, no `.self`, and no key filter (`.enter`) — Draft 0.1's `on event "key" = Action` form is superseded by `event.key` plus a branch inside the action, where the branch is testable at the state layer. Unknown modifiers are `OBIX-T094`.

Delegation. Elements inside `obix:for` receive one delegated listener on the loop container, resolved by walking to the nearest keyed row. A 500-row list attaches one listener, not 500.

13. Event payload normalization

This section is what makes §20 testable, and it is the reason payload handling is specified separately from event mapping.

13.1 The problem

A native `Event` cannot cross the adapter boundary. It is not serializable, it is live (its `target` mutates as the DOM changes), and storing any part of it in state violates Rule J10. But four different adapters must receive **identical payloads from the same user interaction**, or equivalence cannot be asserted against real events.

13.2 The model



```
NativeEvent
  ↓ normalizer (pure, compiler-generated from the binding's PayloadIR)
NormalizedEvent (serializable)
  ↓ extraction
payload : JsonValue[]
  ↓
adapter.dispatch(actionName, ...payload)
```

Rule N1 — the accessor table is closed.

Accessor	Reads	Type
<code>event.value</code>	<code>event.target.value</code>	string
<code>event.checked</code>	<code>event.target.checked</code>	boolean
<code>event.key</code>	<code>event.key</code>	string
<code>event.files</code>	<code>Array.from(event.target.files)</code>	<code>File[]</code> — not storable in state
<code>event.detail</code>	<code>event.detail</code>	any (CustomEvent)
<code>event.x</code> / <code>event.y</code> / <code>event.clientX</code> / <code>event.clientY</code>		number
<code>event.index</code>	loop index at the binding site	number

Anything outside the table is `OBIX-T090`. The raw `Event` object is deliberately unreachable: every legitimate need is either in the table or belongs in an effect.

Rule N2 — no implicit payload. `on:click="Start"` calls `actions.Start(state)`. The event is not passed. There is no default of `event.target.value`, because that default is wrong for `click`, wrong for `change` on a checkbox, wrong for `keydown`, and right only for text inputs. A default that is correct a quarter of the time is worse than no default.

Rule N3 — normalisation happens at the adapter boundary, once. The DOM renderer normalises, then calls `dispatch`. Every adapter beneath receives the same plain array. A Data-adapter test that calls `reduce(state, "SetEmail", "a@b.c")` and a real browser click producing `["a@b.c"]` exercise the identical code path from that point down. This is what lets §23's equivalence blocks assert over recorded interaction sequences and not merely over synthetic ones.

Rule N4 — payloads must be JSON values, except `event.files`, which may be passed to an action but not returned into state (`OBIX-J020` catches the attempt).

Rule N5 — payload arity is capped at 4 (`OBIX-T091`). More arguments suggests the action is doing too much.

13.3 Recorded sequences

Because a `NormalizedEvent` is serializable, an interaction can be recorded as data:

```
[
  { "action": "Start", "payload": [] },
  { "action": "Tick", "payload": [] },
  { "action": "SetLabel", "payload": ["Session 2"] }
]
```

This array is the input to the adapter-equivalence invariant, to telemetry replay, and to the halting/resume model. One representation serves all three.

14. Template bindings

The compiler produces one `BindingIR` per dynamic point in the template, addressed by id, with its dependency set computed at build

time.

Binding kind	Template form	DOM operation
<code>text</code>	<code>{formattedTime}</code> in text	<code>node.textContent = value</code>
<code>attr</code>	<code>aria-valuenow="{seconds}"</code>	<code>setAttribute</code> / <code>removeAttribute</code>
<code>bool</code>	<code>disabled="{isBusy}"</code>	<code>toggleAttribute</code>
<code>aria</code>	<code>aria-expanded="{open}"</code>	<code>setAttribute</code> with <code>"true"</code> / <code>"false"</code> stringification
<code>classPart</code>	<code>class="x x--{status}"</code>	<code>classList.replace</code> on the dynamic segment only
<code>if</code>	<code>obix:if="error"</code>	branch subtree create/remove at an anchor comment
<code>for</code>	<code>obix:for</code> + <code>obix:key</code>	keyed create / move / update / remove
<code>prop</code>	<code>time="{formattedTime}"</code> on a component child prop update, no remount	

14.1 The update path, stated precisely

```

☐
dispatch("Tick")
→ next = actions.Tick(prev)
→ if (Object.is(next, prev)) return // identity: zero work
→ changed = keys where !Object.is(prev[k], next[k]) // ["seconds"]
→ invalidate derived whose stateDeps n changed ≠ ∅ // formattedTime
→ recompute validation rules whose stateDeps n changed ≠ ∅
→ dirty = bindings whose stateDeps n changed ≠ ∅ // b1, b6
→ for each dirty binding: read value; write to DOM only if it differs
→ notify subscribers
```

Three consequences, and they are the whole technical claim:

No virtual DOM. Nothing is diffed. The binding table was computed at build time; the changed-key set indexes straight into it.

No listener re-attachment. DOM nodes are never destroyed by an update, so listeners, focus, scroll position, text selection, and CSS transitions survive. This is the exact failure mode that `innerHTML` re-rendering causes and that the current OBIX architecture forces on every developer.

Minimal work. A `Tick` touches one text node. `statusLabel`, `visualState`, and both `disabled` attributes are not consulted, because `running` did not change.

14.2 Two emitters, one AST

`render` (string) and `mount` (DOM) are generated from the same template AST by different back ends. They cannot drift — server output and client output are provably the same markup because they come from the same tree. This is the structural fix for the SSR problem recorded in the June bottleneck analysis, where documented SSR support and actual `HTMLElement` references had diverged.

15. Accessibility contracts

Accessibility is a compile-time contract, not a runtime library and not an optional lint plugin (I9).

15.1 What the compiler computes per element

Property	Source
<code>role</code>	explicit <code>role</code> , else the tag's implicit ARIA role
<code>name</code>	ACCNAME computation: <code>aria-label</code> , <code>aria-labelledby</code> , associated <code><label></code> , content, <code>alt</code> , <code>title</code>

Property	Source
<code>description</code> <code>aria-describedby</code> target	
<code>focusable</code> interactive tag, or <code>tabindex >= 0</code>	
<code>liveRegion</code> <code>aria-live</code> , or implicit from <code>role="alert"</code> / <code>status</code> / <code>timer</code>	
<code>states</code> bound <code>aria-*</code> attributes and their source bindings	
<code>minBox</code> computed from the scoped stylesheet where statically resolvable	

This computation is possible **only because template expressions are restricted** (§5.3). A template with arbitrary JavaScript in it cannot be analysed this way. That is the payoff for the restriction, and it is why Draft 0.1's instinct to ban expressions was right — Draft 0.2 keeps the ban exactly where it pays and lifts it where it merely cost a standard library.

15.2 Compile-time rules

Rule	Code	Level
Interactive element with no accessible name	<code>OBIX-A001</code>	error
<code></code> without <code>alt</code> (<code>alt=""</code> valid for decorative)	<code>OBIX-A002</code>	error
Form control without label or <code>aria-label</code>	<code>OBIX-A003</code>	error
<code><div></code> / <code></code> with <code>on:click</code> and no role + <code>tabindex</code> + keyboard binding	<code>OBIX-A004</code>	error
<code>aria-*</code> invalid for the element's role	<code>OBIX-A005</code>	error
<code>aria-describedby</code> / <code>aria-labelledby</code> pointing at an absent id	<code>OBIX-A006</code>	error
Heading level skipped within a component	<code>OBIX-A007</code>	warning
Error text not connected to its control	<code>OBIX-A008</code>	warning
Interactive element below 44×44 CSS px in scoped styles	<code>OBIX-A009</code>	warning
Live region with no politeness and no implicit live semantics	<code>OBIX-A010</code>	warning
Positive <code>tabindex</code>	<code>OBIX-A011</code>	warning
<code>autoFocus</code> on more than one element	<code>OBIX-A012</code>	error
Drag-only interaction with no keyboard equivalent	<code>OBIX-A013</code>	warning
Validation rule with <code>field</code> not wired to <code>aria-invalid</code>	<code>OBIX-A014</code>	warning

15.3 FUD policies as compiler passes

The existing Focus / Undo / Drag triad moves from runtime decorators into compilation:

Policy	Compile-time expression
Focus	DOM-order tab order (positive <code>tabindex</code> warned); <code>:focus-visible</code> presence checked in scoped styles; focus preservation <i>guaranteed</i> by keyed list reconciliation rather than left to the developer
Uncertainty / Doubt	Validation declarations (§11) make "was my input accepted, and why not" a declared, wired, asserted part of the component
Undo	Every transition is pure with the previous state still available, so an undo stack is a runtime option (<code>mount(el, { history: n })</code>), not a per-component concern
Drag	Drag-initiating handlers with no non-drag path to the same action are <code>OBIX-A013</code>

A policy checked at build time cannot go stale at runtime — which is the specific defect recorded against the current FUD implementation ("runs once, not at render").

15.4 Keyboard

Rule A20. `on:click` on a native `<button>` or `<a href>` already carries Enter/Space activation from the browser and requires nothing more. On a non-interactive element it requires `role`, `tabindex="0"`, and an explicit `on:keydown`, or `OBIX-A004` fires. OBIX does not synthesise keyboard handlers behind the author's back; it refuses to compile the inaccessible version.

16–19. The DOP adapters

One component definition. Four presentations. The consumer chooses.

```
js
import Timer from "./dist/Timer.js";           // the IR facade (Data adapter)
import { functional, oop, reactive } from "@obinexusltd/obix-dop";

const f = functional(Timer);
const o = oop(Timer);
const r = reactive(Timer);
```

Rule D0 — adapters are wrappers over the Data adapter. Every adapter is defined *in terms of* the reference semantics in §16. No adapter contains a second copy of a transition. This is not a testing convenience; it is what makes §20 provable by construction and testable only for regression.

Rule D1 — adapters may not add, elide, reorder, or coalesce transitions. An adapter may change *how* a transition is requested and *how* the result is observed. It may not change *what* happened.

Rule D2 — no adapter appears in the `.obix` source. A component cannot know, ask, or declare which adapter consumes it (`OBIX-J050`) if `import` from `obix-dop` appears in a component script).

16. Data adapter — the reference semantics

The identity adapter. No instance, no lifecycle, no subscription: pure functions over plain data. It *is* the IR, presented as callable.

```
js
```

```

Timer.state           // initial state literal
Timer.actions.Tick(s) // pure transition
Timer.derived.formattedTime(s, p) // pure projection
Timer.validate(s, p)   // → validity
Timer.render(s, p)     // → HTML string
Timer.reduce(s, "Tick") // → next state
Timer.fold(s, sequence) // → final state

```

```

js
// reference semantics, in full
const reduce = (state, action, ...payload) => Timer.actions[action](state, ...payload);
const fold  = (state, sequence) =>
sequence.reduce((s, step) => reduce(s, step.action, ...(step.payload ?? [])), state);

```

Properties. Stateless, referentially transparent, serializable in and out, runs anywhere (browser, Node, a worker, a test runner, a foreign-language binding reading `Timer.dop.json` and calling into the function table). It is the specification of correctness for the other three.

Use it for: server rendering, contract verification, replaying recorded sequences, telemetry analysis, and any consumer that already has its own state container.

17. Functional adapter

Closure-based, immutable, no `this`.

```

js
const app = functional(Timer, { seconds: 0, running: false });

app.getState();           // → { seconds: 0, running: false }
app.dispatch("Start");    // → { seconds: 0, running: true } (returns the new state)
app.dispatch("SetLabel", "S2"); // payload as trailing arguments
app.derived();            // → { formattedTime: "00:00", statusLabel: "Running", ... }
app.validity();           // → { valid: true, errors: {}, ... }
app.render();             // → HTML string for the current state
app.history();            // → frozen array of states, if enabled

```

```

js
// definition sketch
function functional(C, initial = C.state) {
  let current = Object.freeze(initial);
  return Object.freeze({
    getState: () => current,
    dispatch(action, ...payload) {
      current = Object.freeze(C.reduce(current, action, ...payload));
      return current;
    },
    derived: () => C.deriveAll(current),
    validity: () => C.validate(current),
    render: () => C.render(current)
  });
}

```

Rule F1. `dispatch` returns the new state. Previous states are never mutated; with `{ history: true }` they are retained, which is where undo comes from for free.

18. OOP adapter

Class-instance presentation, generated from the IR. Actions become methods; derived values become getters.

```
js
const timer = oop(Timer);

timer.state;      // frozen plain object
timer.Start();    // method per action; mutates the reference, not the object
timer.Tick();
timer.formattedTime; // getter per derived value
timer.validity;    // getter
timer.render();    // → HTML string
timer.toData();    // → plain state object (the normalisation function for §20)
```

```
js
// definition sketch
function oop(C, initial = C.state) {
  class ObixComponentInstance {
    #state = Object.freeze(initial);
    get state() { return this.#state; }
    get validity() { return C.validate(this.#state); }
    toData() { return this.#state; }
    render() { return C.render(this.#state); }
  }
  for (const a of C.ir.actions) {
    ObixComponentInstance.prototype[a.name] = function (...payload) {
      this.#state = Object.freeze(C.reduce(this.#state, a.name, ...payload));
      return this; // chainable
    };
  }
  for (const d of C.ir.derived) {
    Object.defineProperty(ObixComponentInstance.prototype, d.name, {
      get() { return C.derived[d.name](this.#state, C.props); }
    });
  }
  return new ObixComponentInstance();
}
```

Rule O1 — the object reference is reassigned, never mutated. `this.#state` points at a new frozen object after each method call. This is the crucial distinction from the legacy `ctx.state.x = ...` model (§29): OOP *presentation* is compatible with immutable *semantics*; the two were only ever conflated by accident.

Rule O2 — the bijection is preserved. `toData(oop(C, s)) ≡ functional(C, s).getState()` for every reachable `s`. The DOP adapter's functional $\leftrightarrow$ OOP bijection, already demonstrated at 25/25 in the existing pipeline, generalises here to a four-way equivalence with the Data adapter as the pivot. Every pair is equivalent because every adapter is equivalent to the reference.

Rule O3 — methods are chainable (`timer.Start().Tick().Tick().Stop()`) because the OOP idiom expects it and chaining does not change the transition sequence.

19. Reactive adapter

Observable presentation. This is the adapter the DOM renderer consumes.

```

❏
js
const r = reactive(Timer);

const off = r.subscribe((state, meta) => { /* ... */ });
select("seconds").subscribe((seconds) => { /* ... */ });
dispatch("Start");
getState();
off();           // unsubscribe
dispose();       // HALTED → DESTROYED; stops effects, drops subscribers

```

Rule R1 — one emission per transition, and none for identity returns. If `Object.is(next, prev)` the transition is a no-op and no subscriber is called. This is Rule J23 propagating all the way to the observer layer, and it is the reason a paused `Tick` costs nothing anywhere in the stack.

Rule R2 — emission order is subscription order, synchronous, after validation and before render. No microtask deferral, no batching that could merge two transitions into one emission. Batching would violate D1.

Rule R3 — `select(path)` streams a projection and emits only when that path’s value changes by `Object.is`. Derived names are selectable: `r.select("formattedTime")`.

Rule R4 — `meta` carries the transition record: `{ action, payload, changed: string[] }`. This is exactly the shape the telemetry layer needs, and exactly the shape a recorded sequence (§13.3) is built from.

Rule R5 — the DOM renderer is a consumer, not a fifth adapter. `mount(Timer, el)` is `reactive(Timer)` plus the binding table. Keeping the renderer above the adapter layer is what keeps the adapter layer DOM-free and testable in Node.

20. The Adapter Equivalence invariant

This is the formal centre of Draft 0.2. It is what licenses “there is only `Timer.obix`.”

20.1 Definitions

```

❏

C      a compiled component IR
so    an initial state (JSON value satisfying C.state.schema)
p      a props object
σ      an action sequence [(a1, π1), (a2, π2), ..., (an, πn)]
      where each ai ∈ C.actions and each πi is a JSON payload array

Ref(C, so, σ) = fold(reduce, so, σ)    the reference semantics (§16)
      where reduce(s, (a, π)) = C.actions[a](s, ...π)

norm(A, x) the normalisation of adapter A's observable state to a plain object:
  norm(data, s)    = s
  norm(functional, h) = h.getState()
  norm(oop, i)     = i.toData()
  norm(reactive, r) = r.getState()

x ≡ y    deep structural equality over JSON values

```

20.2 The invariant

E — Adapter Equivalence. For every component `C`, every valid initial state `so`, every props `p`, and every action sequence `σ`, and for every adapter `A ∈ {data, functional, oop, reactive}`:

```
□
norm(A, A.run(C, s0, p, σ)) ≡ Ref(C, s0, σ)
```

Five sub-invariants make E checkable rather than merely asserted:

#	Sub-invariant	Statement
E1	State equivalence	<code>norm(A, ...)</code> $\equiv$ <code>Ref(C, s₀, σ)</code> for all A
E2	Render equivalence	<code>C.render(norm(A,...), p)</code> is byte-identical across all A
E3	Derived & validity equivalence	Every derived value and the whole <code>validity</code> object are <code>≡</code> across all A
E4	Transition-count equivalence	The number of observable transitions is equal across all A. Identity returns (J23) produce zero observable transitions in every adapter — including no <code>reactive</code> emission and no <code>oop</code> state reassignment
E5	Isolation	No adapter mutates <code>s₀</code> or any intermediate state. Verified by deep-freezing every state and running the sequence; a <code>TypeError</code> is a failure
E6	Serialization stability	<code>JSON.parse(JSON.stringify(norm(A,...)))</code> $\equiv$ <code>norm(A,...)</code> for all A

20.3 The worked case

```
□
s0 = { seconds: 0, running: false }
σ = [Start, Tick, Tick, Stop]
```

```
□
Ref: Start → { seconds: 0, running: true }
    Tick → { seconds: 1, running: true }
    Tick → { seconds: 2, running: true }
    Stop → { seconds: 2, running: false }
```

Every adapter must land on `{ seconds: 2, running: false }`:

```
□
js
// Data
Timer.fold(s0, [{action:"Start"},{action:"Tick"},{action:"Tick"},{action:"Stop"}]);

// Functional
const f = functional(Timer, s0);
["Start","Tick","Tick","Stop"].forEach((a) => f.dispatch(a));
f.getState();

// OOP
oop(Timer, s0).Start().Tick().Tick().Stop().toData();

// Reactive
const r = reactive(Timer, s0);
["Start","Tick","Tick","Stop"].forEach((a) => r.dispatch(a));
r.getState();
```

E4 additionally requires that a fifth step `Tick` appended after `Stop` produces **no** observable transition in any adapter, because `Tick` returns `state` unchanged when `running` is false.

20.4 Why this belongs in the conformance artifact, not the behavioural one

Equivalence is a property of the *artifact and its adapters*, not of the developer's feature work. It cannot fail because a feature is half-built; it can only fail because the adapter layer, the IR emission, or a purity violation is broken. That failure must block publication, which is `.obix.test` semantics (I3, §23). Equivalence blocks are therefore part of the contract format and are generated by default for every component with at least one action — a component author gets equivalence coverage without writing it, and may add sequences that exercise domain-specific paths.

20.5 What can break E, in practice

Cause	Detected by
Impure action (mutates state, reads <code>Date.now()</code>)	E1 diverges between adapters; E5 throws
Adapter caching a derived value across a transition	E3
Reactive adapter batching two dispatches into one emission	E4
OOP adapter mutating <code>this.state</code> in place	E5
An action adding a key outside the closed set	E6 or schema check
A legacy <code>ctx</code> -style action reached without the compat wrapper	E5

Each of these is a real defect, and each is caught by a property rather than by a bespoke test someone remembered to write.

21. `Timer.obix` — complete example

html

```
<style lang="css" type="scoped">
.Timer {
  display: grid;
  gap: 1rem;
  justify-items: center;
}

.Timer__display {
  font-variant-numeric: tabular-nums;
  font-size: 3rem;
  line-height: 1;
}

.Timer__display--running {
  color: var(--obix-color-accent, #0b7285);
}

.Timer__controls {
  display: flex;
  gap: 0.5rem;
}

.Timer__button {
  min-inline-size: 3rem;
  min-block-size: 3rem;
  padding: 0.75rem 1.25rem;
}

.Timer__button:focus-visible {
  outline: 3px solid currentColor;
  outline-offset: 2px;
}
</style>

<template>
<section class="Timer" aria-labelledby="Timer-heading">
  <h2 id="Timer-heading" class="Timer__heading">{label}</h2>

  <output
    class="Timer__display Timer__display--{visualState}"
    role="timer"
    aria-live="polite"
    aria-atomic="true">
    {formattedTime}
  </output>

  <p class="Timer__status" aria-live="polite">{statusLabel}</p>

  <div class="Timer__controls">
    <button
      type="button"
      class="Timer__button"
      on:click="Start"
      disabled="{running}">
      Start
    </button>
```

```
<button
  type="button"
  class="Timer__button"
  on:click="Stop"
  disabled="{stopped}">
  Stop
</button>

<button
  type="button"
  class="Timer__button"
  on:click="Reset">
  Reset
</button>
</div>

<p class="Timer__warning" role="status" obix:if="validity.messages.seconds">
  {validity.messages.seconds}
</p>
</section>
</template>

<script>
const props = {
  label: "Timer",
  limitSeconds: 3600,
  idleText: "Ready"
};

const state = {
  seconds: 0,
  running: false
};

const actions = {
  Start(state) {
    if (state.running) return state;
    return { ...state, running: true };
  },

  Stop(state) {
    if (!state.running) return state;
    return { ...state, running: false };
  },

  Reset(state) {
    return { ...state, seconds: 0, running: false };
  },

  Tick(state) {
    if (!state.running) return state;
    return { ...state, seconds: state.seconds + 1 };
  },

  Toggle(state) {
    return state.running ? actions.Stop(state) : actions.Start(state);
  }
}
```

```

};

const derived = {
  formattedTime({ seconds }) {
    const minutes = Math.floor(seconds / 60);
    const remainder = seconds % 60;
    return `${String(minutes).padStart(2, "0")}:${String(remainder).padStart(2, "0")}`;
  },

  statusLabel({ running, seconds }, { idleText }) {
    if (running) return "Running";
    if (seconds > 0) return "Paused";
    return idleText;
  },

  visualState({ running }) {
    return running ? "running" : "idle";
  },

  stopped({ running }) {
    return !running;
  }
};

const validation = {
  withinLimit: {
    field: "seconds",
    severity: "warning",
    message: "Approaching the maximum duration.",
    rule: ({ seconds }, { limitSeconds }) => seconds < limitSeconds * 0.9
  }
};

const effects = {
  tick: {
    every: 1000,
    while: ({ running }) => running,
    dispatch: "Tick"
  }
};
</script>

```

21.1 Evaluation of the supplied design direction

The brief said not to assume every detail was final. Four changes, each with a reason:

Supplied	Specified	Reason
<code>derived.formattedTime(state)</code> taking whole state	<code>formattedTime({ seconds })</code> destructured	Destructuring <i>is</i> the dependency declaration (J41). Whole-state means recompute on every change plus an <code>OBIX-J041</code> advisory.
<code><output aria-live="polite"></code> alone	plus <code>aria-atomic="true"</code> , a heading, and <code>aria-labelledby</code>	<code>role="timer"</code> with <code>aria-live="polite"</code> announces partial updates without <code>aria-atomic</code> ; this is the <code>OBIX-A010</code> neighbourhood.
Stop button always enabled	<code>disabled="{stopped}"</code>	The supplied shape lets you press Stop on a stopped timer. A derived boolean fixes it declaratively and demonstrates T10.
No <code>Tick</code> trigger	<code>const effects</code>	Otherwise the clock never advances without impure code in an action. This is the gap the supplied example leaves open.
Hard-coded label	<code>props.label</code>	Demonstrates the props namespace and makes the component reusable across the clock suite.
<code>Toggle</code> is included to demonstrate S8 (the replacement for <code>choose(running, Stop, Start)</code> in the view) and J24 (actions composing actions).		

22. `Timer.test.obix` — behavioural specification

22.1 Role

Developer-facing TDD. Red before the implementation exists, green after. Exercises state, actions, derived values, validation, effects, and rendered output — the component as a data machine. **Headless: no DOM.**

22.2 The file

html

```
<template>
  <Timer label="Session timer" limitSeconds="5" />
</template>

<script type="test">
  import Timer from "./Timer.obix";

  suite Timer {

    test InitialState "starts stopped at zero" {
      expect state.running == false
      expect state.seconds == 0
      expect derived.formattedTime == "00:00"
      expect derived.statusLabel == "Ready"
    }

    test StartSetsRunning "start begins the timer" {
      when Start
      expect state.running == true
      expect derived.statusLabel == "Running"
    }

    test StartIsIdempotent "starting twice is one transition" {
      when Start
      when Start
      expect state.running == true
      expect transitions == 1
    }

    test TickRequiresRunning "tick does nothing while stopped" {
      given {
        seconds = 4
        running = false
      }
      when Tick
      expect state.seconds == 4
      expect state.unchanged
    }

    test TickAdvances "tick advances while running" {
      given {
        seconds = 4
        running = true
      }
      when Tick
      expect state.seconds == 5
      expect state.running == true
    }

    test EffectDrivesTick "the interval effect ticks once per second" {
      when Start
      advance 3000ms
      expect state.seconds == 3
    }

    test EffectStopsWhenPaused "the effect halts on stop" {
      when Start
```

```
advance 2000ms
when Stop
advance 5000ms
expect state.seconds == 2
}

test ResetReturnsToInitial "reset restores the initial state" {
  when Start
  advance 4000ms
  when Reset
  expect state == initial
}

test MinuteBoundary "formatting crosses the minute boundary" {
  given {
    seconds = 61
  }
  expect derived.formattedTime == "01:01"
}

test ToggleBranches "toggle follows running" {
  when Toggle
  expect state.running == true
  when Toggle
  expect state.running == false
}

test WarningNearLimit "the limit warning fires at 90%" {
  given {
    seconds = 5
  }
  expect validity.warnings.withinLimit != null
  expect render contains "Approaching the maximum duration"
}

test StatePurity "actions do not mutate" {
  given {
    seconds = 10
    running = true
  }
  when Stop
  expect no mutation
}

test Serializable "state survives a JSON round trip" {
  when Start
  expect state serializable
}
}
</script>
```

22.3 Statement forms

Statement	Meaning	Provenance
<code>given { prop = value }</code>	Replace state with the literal, merged over initial	Draft 0.1

Statement	Meaning	Provenance
<code>given props { ... }</code>	Override props for this test	new
<code>when ActionName</code> / <code>when ActionName(arg, ...)</code>	Run an action	Draft 0.1
<code>advance <n>ms</code>	Advance the virtual clock; effects fire deterministically new	
<code>expect <subject> == <value></code> / <code>!=</code> / <code><</code> / <code>></code>	Comparison assertion	Draft 0.1
<code>expect state unchanged</code>	State is identical to before the last <code>when</code>	Draft 0.1 (<code>unchanged</code> keyword)
<code>expect state == initial</code>	Deep equality with the declared initial state	new
<code>expect state serializable</code>	JSON round-trip stable	new
<code>expect no mutation</code>	The pre- <code>when</code> snapshot is byte-identical after	new
<code>expect render contains "<text>"</code>	Substring assertion against <code>render</code>	Draft 0.1 (render layer)
<code>expect transitions == n</code>	Count of <i>observable</i> transitions in this test	new

Subjects: `state`, `state.<path>`, `derived.<name>`, `validity.<path>`, `props.<name>`, `render`, `transitions`.

Note C — `when` **versus** `dispatch`. The Native Component Syntax Proposal used `dispatch`. Draft 0.2 restores Draft 0.1's `when`, because `given` / `when` / `expect` is one recognised idiom and splitting it across two keywords for no semantic gain is churn. `when` as an *action guard* was removed (S4), so there is no collision.

22.4 Semantics

Rule X1 — each `test` gets a fresh instance. Initial state from `Timer.obix`, props from the `<template>` fixture, virtual clock at zero. No shared fixture, no ordering dependence, no `beforeEach`.

Rule X2 — the `<template>` is the fixture. Exactly one component reference with literal props. This is why the behavioural artifact uses the OBIX shape: the thing under test is declared the way it is used.

Rule X3 — virtual time. `advance` steps the effect scheduler, not `Date`. `every: 1000` under `advance 3000ms` dispatches exactly three times, in order, with state settling between each. Effect tests are deterministic and instant.

Rule X4 — headless. No DOM is constructed; `expect render` calls the string emitter. A suite runs identically in Node, a browser, or a CI container — the practical payoff of serializable state.

Rule X5 — layers are implied, not declared. Draft 0.1's `layer state|render|integration` keyword is dropped (S15): a `state.*` subject is a state-layer assertion, a `render` subject is a render-layer assertion, and integration/accessibility assertions live in the other artifact by definition. The concept survives; the ceremony does not.

Rule X6 — the behavioural suite runs against the Data adapter by default. `suite Timer via functional { ... }` runs the same suite through another adapter; equivalence (§20) makes that redundant for correctness, so it exists only for debugging a suspected adapter fault.

Rule X7 — compilation target. A plain ES module exporting a manifest of named test functions, runnable by the small `obix-test` runner (TAP + JSON report) or importable by any host. Not Jest, not Vitest. A thin adapter can expose the manifest to Vitest for teams that want one dashboard — an adapter, not a dependency.

23. `Timer.obix.test` — contract / conformance specification

23.1 Role

Machine-verifiable contract attached to the artifact. Consumer-facing and tooling-facing. Portable: it can be run against *any* implementation claiming to be `Timer` — which is Draft 0.1's framing (I4), preserved exactly.

It is not a three-section file. It has no style and no template, because it is not a component; giving it the component shape would imply it can render, which it cannot. It is a flat declarative contract.




```
sequence Tick, Tick, Tick
expect state { seconds: 2, running: false }
expect transitions == 0
across data, functional, oop, reactive
}

element Display {
  select role "timer"
  expect aria-live "polite"
  expect aria-atomic "true"
  expect label from content
  expect text matches "[0-9]{2};[0-9]{2}$"
}

element StartButton {
  select role "button" label "Start"
  expect focusable
  expect touch-target 44x44
  expect keyboard Enter dispatches Start
  expect keyboard Space dispatches Start
  expect focus-visible
}

element StopButton {
  select role "button" label "Stop"
  given { running = false }
  expect aria disabled true
  given { running = true }
  expect aria disabled false
}

region Component {
  expect heading-order valid
  expect no positive-tabindex
  expect tab-order [StartButton, StopButton, ResetButton]
  expect contrast 4.5
}

announce {
  given { running = false, seconds = 0 }
  when Start
  advance 1000ms
  expect live-region Display announces "00:01"
```

23.2 Blocks

Block	Asserts	Draft 0.1 keyword reused
<code>surface { ... }</code>	The published API: props, state keys with types and defaults, action names, derived names and return types, validation rules. Any rename or removal is a contract break.	<code>contract</code>
<code>invariant Name { ... }</code>	A property holding for all reachable states, verified over declared cases and, where the state space is enumerable, by exploring the action graph.	<code>invariant</code>
<code>contract Action { pre ... post ... }</code>	Pre/postconditions on a single transition. <code>old state.x</code> refers to the pre-transition value.	<code>pre</code> , <code>post</code> , <code>old</code> , <code>unchanged</code>


```
{
  "name": "Timer",
  "irVersion": "1.0",

  "metadata": {
    "source": "src/Timer.obix",
    "compiler": "obixc 1.0.0",
    "hash": "sha256:7f3c...",
    "scopeToken": "Timer",
    "lifecycle": ["CREATED", "UPDATED", "HALTED", "DESTROYED"],
    "props": [
      { "name": "label",    "type": "string", "default": "Timer" },
      { "name": "limitSeconds", "type": "number", "default": 3600 },
      { "name": "idleText",  "type": "string", "default": "Ready" }
    ],
    "imports": []
  },

  "state": {
    "initial": { "seconds": 0, "running": false },
    "keys": ["seconds", "running"],
    "schema": {
      "seconds": { "type": "number", "nullable": false, "default": 0 },
      "running": { "type": "boolean", "nullable": false, "default": false }
    },
    "serializable": true
  },

  "actions": [
    { "name": "Start", "arity": 1, "params": ["state"], "payload": [],
      "fn": "actions.Start", "identityPaths": true,
      "pure": { "mutations": 0, "globals": [], "async": false } },
    { "name": "Stop", "arity": 1, "params": ["state"], "payload": [],
      "fn": "actions.Stop", "identityPaths": true,
      "pure": { "mutations": 0, "globals": [], "async": false } },
    { "name": "Reset", "arity": 1, "params": ["state"], "payload": [],
      "fn": "actions.Reset", "identityPaths": false,
      "pure": { "mutations": 0, "globals": [], "async": false } },
    { "name": "Tick", "arity": 1, "params": ["state"], "payload": [],
      "fn": "actions.Tick", "identityPaths": true,
      "pure": { "mutations": 0, "globals": [], "async": false } },
    { "name": "Toggle", "arity": 1, "params": ["state"], "payload": [],
      "fn": "actions.Toggle", "identityPaths": false,
      "pure": { "mutations": 0, "globals": [], "async": false },
      "calls": ["Start", "Stop"] }
  ],

  "derived": [
    { "name": "formattedTime", "stateDeps": ["seconds"], "propDeps": [], "fn": "derived.formattedTime", "returns": "string" },
    { "name": "statusLabel", "stateDeps": ["running", "seconds"], "propDeps": ["idleText"], "fn": "derived.statusLabel", "returns": "string" },
    { "name": "visualState", "stateDeps": ["running"], "propDeps": [], "fn": "derived.visualState", "returns": "string" },
    { "name": "stopped", "stateDeps": ["running"], "propDeps": [], "fn": "derived.stopped", "returns": "boolean" }
  ],

  "validation": [
    { "name": "withinLimit", "field": "seconds", "severity": "warning",
      "stateDeps": ["seconds"], "propDeps": ["limitSeconds"],
```

```
"message": "Approaching the maximum duration.",
"fn": "validation.withinLimit.rule" }
],

"effects": [
{ "name": "tick", "kind": "every", "value": 1000,
  "whileDeps": ["running"], "whileFn": "effects.tick.while",
  "dispatch": "Tick", "payload": [] }
],

"render": {
  "emitters": ["dom", "string"],
  "anchors": [ { "id": "a0", "kind": "if", "binding": "b6" } ],
  "bindings": [
    { "id": "b0", "kind": "text",    "target": "0/0/0",
      "read": { "ns": "props", "path": "label" },
      "stateDeps": [], "propDeps": ["label"] },
    { "id": "b1", "kind": "text",    "target": "0/1/0",
      "read": { "derived": "formattedTime" },
      "stateDeps": ["seconds"], "propDeps": [] },
    { "id": "b2", "kind": "classPart", "target": "0/1",
      "read": { "derived": "visualState" }, "prefix": "Timer__display--",
      "stateDeps": ["running"], "propDeps": [] },
    { "id": "b3", "kind": "text",    "target": "0/2/0",
      "read": { "derived": "statusLabel" },
      "stateDeps": ["running", "seconds"], "propDeps": ["idleText"] },
    { "id": "b4", "kind": "bool",     "target": "0/3/0", "attribute": "disabled",
      "read": { "ns": "state", "path": "running" },
      "stateDeps": ["running"], "propDeps": [] },
    { "id": "b5", "kind": "bool",     "target": "0/3/1", "attribute": "disabled",
      "read": { "derived": "stopped" },
      "stateDeps": ["running"], "propDeps": [] },
    { "id": "b6", "kind": "if",      "target": "0/@a0",
      "read": { "validity": "messages.seconds" },
      "stateDeps": ["seconds"], "propDeps": ["limitSeconds"] }
  ],
  "static": "<section class=\"Timer\" data-obix=\"Timer\" ...>...</section>"
},

"events": [
{ "id": "e0", "target": "0/3/0", "type": "click", "modifiers": [],
  "action": "Start", "payload": [], "delegated": false },
{ "id": "e1", "target": "0/3/1", "type": "click", "modifiers": [],
  "action": "Stop", "payload": [], "delegated": false },
{ "id": "e2", "target": "0/3/2", "type": "click", "modifiers": [],
  "action": "Reset", "payload": [], "delegated": false }
],

"accessibility": {
  "nodes": [
    { "nodeId": "0",    "role": "region", "roleImplicit": false,
      "name": { "source": "aria", "ref": "Timer-heading" }, "focusable": false },
    { "nodeId": "0/1",  "role": "timer", "roleImplicit": false,
      "name": { "source": "content", "binding": "b1" }, "focusable": false },
    { "nodeId": "0/3/0", "role": "button", "roleImplicit": true,
      "name": { "source": "content", "text": "Start" }, "focusable": true,
      "keyboard": [ { "key": "Enter", "action": "Start" },
```

```
{
  { "key": "Space", "action": "Start" } ],
  "minBox": { "width": 48, "height": 48 },
  "states": [ { "attribute": "disabled", "binding": "b4" } ] ]
},
"tabOrder": [ "0/3/0", "0/3/1", "0/3/2" ],
"headings": [ { "level": 2, "nodeId": "0/0" } ],
"liveRegions": [
  { "nodeId": "0/1", "politeness": "polite", "atomic": true },
  { "nodeId": "0/2", "politeness": "polite", "atomic": false }
],
"idRefs": [
  { "from": "0", "attribute": "aria-labelledby", "to": "Timer-heading", "resolved": true }
]
}
```

What this buys. A tool that has never executed a line of `Timer.js` can read this file and answer: what are the state keys and their types; which actions exist and with what arity; which state key drives which DOM operation; what the accessible name of every control is; whether the tab order is DOM order; which validation rules exist and what they are about. That is why the declarations are data. It is also the interchange point for a polyglot binding: a Go or Rust consumer reads the JSON and calls into the ES function table across the boundary, without needing to parse `.obix` at all.

25. Generated native ES6

`dist/Timer.js` — the function table plus the two emitters plus the adapter facade. Abridged; structurally faithful.

js

```
// dist/Timer.js — generated by obixc 1.0.0 from src/Timer.obix
// @generated — do not edit
```

```
import ir from "./Timer.dop.json" with { type: "json" };
import { bindText, bindBool, bindClassPart, bindEvent, createlfBlock,
  createInstance, mountEffects } from "@obinexusltd/obix-runtime";

/* ---- 1. contract bindings, verbatim from <script> ---- */

export const props = { label: "Timer", limitSeconds: 3600, idleText: "Ready" };

export const state = { seconds: 0, running: false };

export const actions = {
  Start(state) { if (state.running) return state; return { ...state, running: true }; },
  Stop(state) { if (!state.running) return state; return { ...state, running: false }; },
  Reset(state) { return { ...state, seconds: 0, running: false }; },
  Tick(state) { if (!state.running) return state; return { ...state, seconds: state.seconds + 1 }; },
  Toggle(state) { return state.running ? actions.Stop(state) : actions.Start(state); }
};

export const derived = {
  formattedTime({ seconds }) {
    const minutes = Math.floor(seconds / 60);
    const remainder = seconds % 60;
    return `${String(minutes).padStart(2, "0")}:${String(remainder).padStart(2, "0")}`;
  },
  statusLabel({ running, seconds }, { idleText }) {
    if (running) return "Running";
    if (seconds > 0) return "Paused";
    return idleText;
  },
  visualState({ running }) { return running ? "running" : "idle"; },
  stopped({ running }) { return !running; }
};

export const validation = {
  withinLimit: {
    field: "seconds", severity: "warning",
    message: "Approaching the maximum duration.",
    rule: ({ seconds }, { limitSeconds }) => seconds < limitSeconds * 0.9
  }
};

export const effects = {
  tick: { every: 1000, while: ({ running }) => running, dispatch: "Tick" }
};

/* ---- 2. reference semantics (the Data adapter) ---- */

export function reduce(currentState, action, ...payload) {
  const fn = actions[action];
  if (!fn) throw new TypeError(`Timer: no action "${action}"`);
  return fn(currentState, ...payload);
}

export function fold(initial, sequence) {
```

```

return sequence.reduce(
  (s, step) => reduce(s, step.action, ...(step.payload ?? [])),
  initial
);
}

export function deriveAll(s, p = props) {
  return {
    formattedTime: derived.formattedTime(s, p),
    statusLabel: derived.statusLabel(s, p),
    visualState: derived.visualState(s, p),
    stopped: derived.stopped(s, p)
  };
}

export function validate(s, p = props) {
  const errors = {}, warnings = {}, invalid = {}, messages = {};
  let valid = true;
  for (const [name, rule] of Object.entries(validation)) {
    const ok = rule.rule(s, p);
    const bucket = rule.severity === "warning" ? warnings : errors;
    bucket[name] = ok ? null : rule.message;
    if (!ok && rule.severity !== "warning") valid = false;
    if (rule.field) {
      invalid[rule.field] = invalid[rule.field] || !ok;
      messages[rule.field] = messages[rule.field] || (ok ? null : rule.message);
    }
  }
  return { valid, errors, warnings, invalid, messages };
}

/* ---- 3. string emitter (SSR) ---- */

export function render(s = state, p = props) {
  const v = validate(s, p);
  const esc = (x) => String(x).replace(/([<>"])/g, (c) =>
    ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c]));

  return `<section class="Timer" data-obix="Timer" aria-labelledby="Timer-heading">` +
    `<h2 id="Timer-heading" class="Timer__heading" data-obix="Timer">${esc(p.label)}</h2>` +
    `<output class="Timer__display Timer__display--${esc(derived.visualState(s, p))}"` +
    `role="timer" aria-live="polite" aria-atomic="true" data-obix="Timer">` +
    `${esc(derived.formattedTime(s, p))}</output>` +
    `<p class="Timer__status" aria-live="polite" data-obix="Timer">${esc(derived.statusLabel(s, p))}</p>` +
    `<div class="Timer__controls" data-obix="Timer">` +
    `<button type="button" class="Timer__button" data-obix="Timer"${s.running ? " disabled" : ""}>Start</button>` +
    `<button type="button" class="Timer__button" data-obix="Timer"${derived.stopped(s, p) ? " disabled" : ""}>Stop</button>` +
    `<button type="button" class="Timer__button" data-obix="Timer">Reset</button>` +
    `</div>` +
    `(v.messages.seconds` +
    ` ? `<p class="Timer__warning" role="status" data-obix="Timer">${esc(v.messages.seconds)}</p>` +
    ` : ""` +
    `</section>`;
}

export const renderToString = render;

```

```

/* ---- 4. DOM emitter (consumes the reactive adapter) ---- */

const TEMPLATE = /* @__PURE__ */ (() => {
  const t = document.createElement("template");
  t.innerHTML = ir.render.static;
  return t;
})();

export function mount(target, options = {}) {
  const instance = createInstance({ ir, actions, derived, validation, effects,
    props: { ...props, ...(options.props || {}) } });

  const root = TEMPLATE.content.cloneNode(true);
  const section = root.firstElementChild;
  const at = (path) => instance.resolve(section, path); // "0/3/1" → node

  instance.bindings = [
    bindText(instance, "b0", at("0/0/0"), (s, p) => p.label),
    bindText(instance, "b1", at("0/1/0"), () => instance.compute("formattedTime")),
    bindClassPart(instance, "b2", at("0/1"), "Timer__display--",
      () => instance.compute("visualState")),
    bindText(instance, "b3", at("0/2/0"), () => instance.compute("statusLabel")),
    bindBool(instance, "b4", at("0/3/0"), "disabled", (s) => s.running),
    bindBool(instance, "b5", at("0/3/1"), "disabled", () => instance.compute("stopped")),
    createIfBlock(instance, "b6", at("0/@a0"), () => instance.validity().messages.seconds)
  ];

  bindEvent(instance, at("0/3/0"), "click", "Start");
  bindEvent(instance, at("0/3/1"), "click", "Stop");
  bindEvent(instance, at("0/3/2"), "click", "Reset");

  instance.flushAll(); // CREATED
  target.appendChild(root);
  mountEffects(instance); // starts/stops `tick` per its `while` predicate
  return instance; // .state.dispatch().subscribe().unmount()
}

export function hydrate(element, options = {}) { /* adopt existing DOM, bind, no re-render */ }

/* ---- 5. the Data adapter facade — the default export ---- */

export default {
  name: "Timer", ir,
  props, state, actions, derived, validation, effects,
  reduce, fold, deriveAll, validate,
  render, renderToString, mount, hydrate
};

```

Note. `functional`, `oop`, and `reactive` are *not* in this file. They live in `@obinexusltd/obix-dop` and take this default export as input. That is the structural expression of "one component definition, four presentations": the component module contains no paradigm.

26. AST

The compiler builds one tree per file. The template is never a string.



```
ObixComponent {
  name, filename, scopeToken
  styles      : StyleSection[]
  template    : TemplateSection
  script      : ScriptSection | null
  imports     : ImportDecl[]
  props       : PropDecl[]
  state       : StateDecl[]
  actions     : ActionDecl[]
  derived     : DerivedDecl[]
  validation  : ValidationDecl[]
  effects     : EffectDecl[]
  bindings    : BindingRecord[]
  accessibility : A11yModel
  diagnostics : Diagnostic[]
  sourceMap   : SourceMapIndex
}
```

Sections

```
StyleSection { lang, type, source, rules: CssRule[], span }
TemplateSection { children: TemplateNode[], span }
ScriptSection { lang, type, esAst: EstreeProgram, span }
```

Template nodes

```
TemplateNode = Element | Component | Text | Interpolation
              | Conditional | Loop | Slot | Comment

Element      { tag, attributes: Attribute[], events: EventBinding[],
              children: TemplateNode[], selfClosing, a11y: A11yNode, span }
Component    { tag, resolvedFrom: ImportDecl, props: Attribute[],
              events: EventBinding[], slotContent: SlotContent[], span }
Text         { value, span }
Interpolation { path: Path, bindingId, span }

Attribute    { name, value: AttributeValue, isBoolean, isAria, span }
AttributeValue = LiteralValue { text }
              | BoundValue    { path, bindingId }
              | MixedValue    { parts: (string | Interpolation)[], bindingId }

EventBinding { event, modifiers[], action, args: EventArg[], delegated, span }
EventArg     = EventAccessor { accessor } | PathArg { path } | LiteralArg { value }

Conditional { branches: [{ test: Path|null, node }], anchorId, span }
Loop        { itemName, indexName, source: Path, key: Path, node, anchorId, scope, span }
Slot        { name, fallback: TemplateNode[], span }
SlotContent { slot, nodes, owner: "parent" }

Path         { head, segments[], namespace: "loop"|"props"|"derived"|"state"|"validity",
              qualified, span }
```

Script declarations

```
PropDecl    { name, defaultValue, inferredType, span }
StateDecl   { name, initialValue, valueType, serializable, span }
ActionDecl  { name, arity, paramNames, purity: PurityReport, identityPaths, calls[], span }
DerivedDecl { name, stateDeps: string[]"\"", propDeps: string[]"\"", span }
ValidationDecl { name, field, severity, message, stateDeps, propDeps, span }
EffectDecl  { name, kind, value, whileDeps, dispatch, args, span }
ImportDecl  { local, source, isObixComponent, span }
```

Accessibility model

```
A11yModel { nodes: A11yNode[], tabOrder[], headings[], liveRegions[], idRefs[] }
A11yNode  { nodeId, role, roleImplicit, name: { text|path, source },
            description, focusable, tabIndex, states[], keyboard[], minBox }
```

Binding record — the compile-time output that drives updates:

```
BindingRecord { id, kind, target: NodePath,
                read: Path | DerivedRef | ValidityRef,
                stateDeps: string[]"\"", propDeps: string[]"\""
```

`stateDeps` is the entire fine-grained update mechanism, computed statically before a byte ships. Nothing observes anything at runtime.

27. EBNF

ISO-style. `=` defines, `,` concatenates, `|` alternates, `{ }` zero-or-more, `[ ]` optional, `-` excludes. LL(1) at every decision point after tokenisation; implementable by hand-written recursive descent.

27.1 File and sections

```
ebnf
ObixFile    = { WS }, { StyleSection, { WS } },
              TemplateSection, { WS },
              [ ScriptSection ], { WS }, EOF ;

StyleSection = "<style", { SectionAttribute }, ">", StyleBody, "</style>";
TemplateSection = "<template", ">", TemplateBody, "</template>";
ScriptSection = "<script", { SectionAttribute }, ">", ScriptBody, "</script>";

SectionAttribute= WS+, AttrName, "=", "'", AttrValue, "'";
StyleBody      = { AnyChar - "</style>" };
ScriptBody     = { AnyChar - "</script>" };
TemplateBody   = { TemplateNode };
```

`StyleBody` goes to a CSS parser; `ScriptBody` to a standard ES parser, or — when `type="test"` — to §27.5.

27.2 Template

```
ebnf
```

```
TemplateNode = Element | ComponentReference | SlotElement | TextRun | Comment ;
```

```
Element      = "<", ElementName, { WS+, Attribute }, { WS },  
              ( ">" | ">", { TemplateNode }, "</", ElementName, ">" );
```

```
ComponentReference  
    = "<", ComponentName, { WS+, Attribute }, { WS },  
      ( ">" | ">", { TemplateNode }, "</", ComponentName, ">" );
```

```
SlotElement  = "<slot", [ WS+, "name", "=", "", Identifier, "" ], { WS },  
              ( ">" | ">", { TemplateNode }, "</slot>" );
```

```
ElementName  = LowerLetter, { LowerLetter | Digit | "-" } ;
```

```
ComponentName = UpperLetter, { Letter | Digit } ;
```

```
TextRun      = TextChunk, { TextChunk } ;
```

```
TextChunk    = Interpolation | LiteralText ;
```

```
LiteralText  = ( AnyChar - "<" - "{" | EscapedBrace )+ ;
```

```
EscapedBrace = "{ " | " }" ;
```

```
Comment      = "<!--", { AnyChar - "-->" }, "-->" ;
```

27.3 Attributes, paths, bindings, directives



ebnf

```
Attribute    = EventBinding | StructuralDirective | NormalAttribute ;
```

```
NormalAttribute = AttrName, [ "=", "", AttributeValue, "" ] ;
```

```
AttrName     = ( Letter | "_" ), { Letter | Digit | "-" | "_" | ":" } ;
```

```
AttributeValue = { AttrChunk } ;
```

```
AttrChunk    = Interpolation | AttrLiteral ;
```

```
AttrLiteral  = ( AnyChar - "'" - "{" | EscapedBrace )+ ;
```

```
Interpolation = "{", { WS }, Path, { WS }, "}" ;
```

```
Path         = [ Namespace, "." ], Identifier, { PathSegment } ;
```

```
Namespace    = "state" | "props" | "derived" | "validity" ;
```

```
PathSegment  = ".", Identifier | ".", DigitSequence | "[", DigitSequence, "]" ;
```

```
EventBinding = "on:", EventType, { ".", Modifier }, "=", "", ActionCall, "" ;
```

```
EventType    = LowerLetter, { LowerLetter | Digit | "-" } ;
```

```
Modifier     = "prevent" | "stop" | "once" | "capture" ;
```

```
ActionCall   = ActionName, [ { WS }, "(", { WS }, [ ArgumentList ], { WS }, ")" ] ;
```

```
ActionName   = UpperLetter, { Letter | Digit } ;
```

```
ArgumentList = Argument, { { WS }, ",", { WS }, Argument } ;
```

```
Argument     = EventAccessor | Path | Literal ;
```

```
EventAccessor = "event.", ( "value" | "checked" | "key" | "files"  
                           | "detail" | "x" | "y" | "index" ) ;
```

```
StructuralDirective
```

```
    = "obix:if",      "=", "", Path, ""  
    | "obix:else-if", "=", "", Path, ""  
    | "obix:else"  
    | "obix:for",     "=", "", LoopHeader, ""  
    | "obix:key",     "=", "", Path, ""  
    | "obix:slot",    "=", "", Identifier, "" ;
```

```
LoopHeader    = Identifier, [ { WS }, ",", { WS }, Identifier ], WS+, "of", WS+, Path ;
```

27.4 Lexical



ebnf

27.5 Behavioural test DSL (`.test.obix`)



```
ebnf
```

27.6 Contract DSL (`.obix.test`)



```
ebnf
```



```
| "expect", "contrast", NumberLiteral ;
```

```
AnnounceBlock = "announce", "{", { TestStatement },  
"expect", "live-region", Identifier, "announces", StringLiteral, "}" ;
```

27.7 Grammar properties

Element vs component: decided by the first character after `<`. One character of lookahead.

Literal text vs binding: decided by `{`, escaped as `\{`. One character of lookahead.

No expression grammar in the template: `Path` is a straight-line production with no recursion into operators. There is no precedence table because there are no operators. This is the property that makes static accessibility analysis, dependency graphs, and SSR safety possible — and it is exactly the restriction Draft 0.1 got right.

The script body is not in this grammar. It is delegated to a standard ES parser. OBIX never forks JavaScript.

28. Compiler pipeline





28.1 Notes

Stage 9 is the architectural centre of Draft 0.2. Everything before it is source analysis; everything after it is a projection of the IR. The adapters do not read `.obix` and do not read the AST — they read the IR facade. That is what makes "one component definition, four presentations" structural rather than aspirational.

Stage 7 is where the performance claim is earned. By the end of it the compiler knows, for every state key, the exact set of DOM operations that key can cause. Nothing is discovered at runtime. This is the tennis-tracker state-minimisation principle applied to rendering: enumerate the transitions that exist, eliminate the rest, never pay for a path that cannot be taken.

Stage 10 makes the contract a build input. Surface drift fails compilation before any test runs.

Two emitters, one AST (stage 11). Server and client output come from the same tree and cannot diverge.

28.2 CLI

```
bash
obixc build src --out dist      # compile: .js + .dop.json + .css
obixc build src --emit-ir-only  # IR only, for tooling and foreign bindings
obixc test src                  # run *.test.obix (headless, virtual clock)
obixc verify src                # run *.obix.test (contract + equivalence + DOM)
obixc contract --emit src/Timer.obix # generate a starter Timer.obix.test from the IR
obixc check src                 # diagnostics only, no emit
obixc explain OBIX-T030         # long-form diagnostic documentation
```

`obixc verify` is the publication gate. Orchestration (`nlink` → `polybuild`) runs `build` then `verify`; a failing contract stops the artifact from advancing.

29. Legacy DOPAdapter compatibility

The existing prototype's transitions mutate a context object:

```
js
// legacy component logic
const ButtonLogic = {
  state: { clicked: false },
  actions: {
    toggle(ctx) { ctx.state.clicked = !ctx.state.clicked; }
  }
};
```

This model is **supported, deprecated, and quarantined at the boundary**. It is not the language semantics and it never becomes the language semantics.

29.1 Inbound: legacy logic → canonical DOP

```
js
import { fromLegacy } from "@obinexusltd/obix-dop/compat";

const Button = fromLegacy(ButtonLogic); // → a canonical DOP component
Button.reduce({ clicked: false }, "toggle"); // → { clicked: true }
```

The wrapper converts mutation into a pure transition at the boundary:

```
js
function liftLegacyAction(legacyFn) {
  return function (state, ...payload) {
    const ctx = { state: structuredClone(state) }; // draft
    legacyFn(ctx, ...payload); // legacy mutates the draft
    return Object.freeze(ctx.state); // snapshot as the new state
  };
}
```

Rules.

#	Rule
L1	The draft is a structured clone. The caller's state is never reachable by the legacy handler.
L2	Legacy handlers must be synchronous. An async handler is <code>OBIX-L002</code> — the snapshot would be taken before the mutation.
L3	The result is frozen and validated against the serializability rule. A DOM node mutated into <code>ctx.state</code> fails here rather than three layers downstream.
L4	A legacy handler's return value is ignored; the draft is the truth. Handlers that return state are <code>OBIX-L003</code> (probably already migrated — remove the wrapper).
L5	Action names are lifted verbatim, so <code>toggle</code> stays <code>toggle</code> . Renaming to <code>Toggle</code> is a migration step, not something the wrapper does silently.
L6	Every lift emits <code>OBIX-L001</code> (deprecation) naming the action and the file.

Cost, stated honestly. `structuredClone` per transition is real overhead, proportional to state size. Canonical actions have none. This is a deliberate asymmetry: the compatibility path works, and it is measurably slower than migrating.

29.2 Outbound: canonical component → legacy consumer

Existing code that consumes `new DOPAdapter(logic).toFunctional()` keeps working:

```
import { toLegacy } from "@obinexusltd/obix-dop/compat";

const legacyShape = toLegacy(Timer); // { state, actions: ctx-style, render }
const adapter = new DOPAdapter(legacyShape);
adapter.toFunctional();
adapter.toOOP();
```

`toLegacy` wraps each pure action in a ctx-mutating shim that assigns the returned state onto `ctx.state`. The transition is still pure underneath; the mutation is a presentation detail applied at the very edge, which is the same relationship §18 has with OOP.

29.3 Equivalence under compatibility

Rule L7. A lifted legacy component must satisfy the full §20 invariant. `fromLegacy(X)` produces a component whose `.obix.test` includes generated equivalence blocks like any other. This is the practical value of the compat layer beyond keeping code alive: **it is a migration verifier**. Lift the legacy component, run equivalence across all four adapters, and any hidden impurity — a shared mutable closure, a `Date.now()` in a handler, a nested mutation — shows up as a divergence rather than as a bug six months later.

29.4 Lifecycle

Phase	Status
1.0	Supported. Deprecation warning at every lift.
1.x	Supported. <code>obixc migrate --legacy</code> generates canonical <code><script></code> skeletons from legacy logic.
2.0	Removed. <code>fromLegacy</code> / <code>toLegacy</code> move to a separately versioned package or are dropped.

30. Unresolved questions

Each is recorded with what is known, what is blocking, and what would unblock it.

Q1 — Child → parent semantic events. Native bubbling covers `click` reaching a parent but not “the child finished loading”. A pure action cannot dispatch a `CustomEvent`. Two candidates: (a) actions return `[nextState, emissions]` — expressive, but complicates every signature, every test assertion, and the equivalence invariant (which would need to compare emission traces as well as state); (b) a

declarative `const emits = { done: { when: ({ finished }) => finished } }` where the runtime dispatches on predicate transition, leaving actions untouched. (b) fits the effects model and does not disturb §20. **Blocking:** whether emissions need payloads richer than a state snapshot.

Q2 — Asynchronous data. `fetch` needs pending/success/error, cancellation on unmount, and race resolution. Candidate: an effect kind `request` with three dispatch targets (`onStart`, `onSuccess`, `onError`). **Blocking:** whether OBIX should own request lifecycle at all, or whether data belongs above the component in an application shell. Deciding this wrongly puts a network stack inside a UI language. Note the interaction with §20: async introduces non-determinism into the action sequence, so the equivalence invariant would need a scheduler model.

Q3 — Prop spreading. `<Child {...config} />` is convenient and defeats static prop checking (T70). Options: forbid (current position), or permit only where the spread source is a statically analysable literal. **Blocking:** real usage data.

Q4 — `type="module"` styles. Reserved (§4.3). Reopens only if attribute scoping proves insufficient — most plausibly for dynamic class selection from state, which a `class:name="{path}"` directive might address without CSS Modules at all. **Blocking:** a concrete failing case.

Q5 — Two-way binding shorthand. `bind:value="email"` would compile to exactly `value="{email}" on:input="SetEmail(event.value)"`. Sugar with no new semantics, therefore safe — but it adds a third directive prefix and a name that elsewhere implies mutation, which OBIX does not have. **Blocking:** naming, and whether the verbosity is a real problem.

Q6 — Nested state granularity. Rule J13 compares top-level keys by identity, so any change under `settings` re-runs every binding under `settings`. Options: path-level diffing at a declared depth, or a flat-state convention. **Blocking:** measurement; this may never be a real cost.

Q7 — Validation severity and transition blocking. Rule V1 makes validation purely descriptive. A future `severity: "blocking"` would let a rule reject a transition — attractive for forms, but it turns `reduce` into a partial function and complicates §20 (adapters would have to agree on rejection, not just on results). **Blocking:** whether the identity-return idiom inside actions is genuinely insufficient in practice.

Q8 — Adapter-specific IR extensions. Should the reactive adapter be allowed to read IR fields the others ignore (for example, a declared emission cadence)? Currently no adapter reads anything the others cannot. Relaxing this is how adapters begin to diverge. **Blocking:** a use case strong enough to justify the risk.

Q9 — Contract inheritance. Should `AccessibleButton.obix.test` be able to `extends "../BaseButton.obix.test"` so a component family shares one accessibility contract? Attractive for the 30-component library; adds a resolution order to the contract format. **Blocking:** whether those 30 components genuinely share contract shape or only appear to.

Q10 — Legacy interop for existing runtime components. Compiled `.obix` output is shape-compatible with today's `{ state, actions, render }` components, so `.obix` output drops into the existing runtime. The reverse — using a hand-written `obix-component-runtime` component as a `<Tag />` in a template — needs a declared prop surface those components do not have. Candidate: generate a `surface` `{}` block per legacy component, which would also retrofit the contract layer onto the existing library. **Blocking:** effort estimate across 30 components.

Q11 — Animation on conditional and loop exit. Removing a node cancels exit animations. A declarative `obix:transition` is a substantial feature and may be unnecessary now that browsers support `@starting-style` and `transition-behavior: allow-discrete`. **Blocking:** browser support survey.

Q12 — OBIXverse. Application suites, reverse-domain identities (`com.myclock.timer`), routing, manifests, installability. Deliberately undesigned. Two constraints are recorded because they are cheap now and expensive to retrofit: (1) a component must be addressable by reverse-domain identity without changing its source — identity belongs to the application manifest, not the `.obix` file; (2) serializable state is the interchange format between OBIXverse applications, which the state rules already guarantee.

Appendix A — Migration table: Draft 0.1 → Draft 0.2

#	Draft 0.1	→	Draft 0.2	Reason
1	<code>component Timer { ... }</code>	→ filename	<code>Timer.obix</code>	One source of truth (§3.1)
2	<code>state { seconds: number = 0 }</code>	→	<code><script> const state = { seconds: 0 };</code>	Serializable literal; no bespoke declaration grammar (§3.2)

#	Draft 0.1	→	Draft 0.2	Reason
3	<code>seconds: number</code> type annotation	→	<code>inferred from the literal; declared in <code>surface { state seconds: number = 0 }</code></code>	Types where they are contractual (§3.2)
4	<code>action Start { running -> true }</code>	→	<code>actions.Start(state) { return { ...state, running: true }; }</code>	<code>-></code> cannot express real transitions (§3.3)
5	<code>action Tick { when running ... }</code>	→	<code>if (!state.running) return state;</code>	Identity return is also the zero-work signal (§3.3, J23)
6	<code>propertyName -> expression</code>	→	object spread returning a new state	Immutability is the semantics, not a convention (§9)
7	<code>view { ... }</code>	→	<code><template> ... </template></code>	Real HTML keeps the accessibility toolchain (§3.4)
8	<code>div timerDisplay { ... }</code>	→	<code><div id="timerDisplay"></code>	Element identity is an HTML id
9	<code>text = formatTime(seconds)</code>	→	<code>{formattedTime}</code> + <code>const derived</code>	Paths in templates; logic in script (§5.3, §10)
10	<code>attribute = expression</code>	→	<code>attribute="{path}"</code>	Restricted, statically resolvable (§5.5)
11	<code>aria-live = "polite"</code>	→	<code>aria-live="polite"</code>	Native HTML attribute
12	<code>label = "Reset"</code>	→	element text content, or <code>aria-label</code>	<code>label</code> was ambiguous (§3.4)
13	<code>on click = Reset</code>	→	<code>on:click="Reset"</code>	Direct <code>addEventListener</code> mapping (§12)
14	<code>on click = choose(running, Stop, Start)</code>	→	<code>on:click="Toggle"</code> with the branch in the action	Static event→action map; testable at state layer (§3.5)
15	<code>on event "key" = Action</code>	→	<code>on:keydown="HandleKey(event.key)"</code>	Branch in the action, where it is testable (§12)
16	<code>choose(cond, a, b)</code> in view	→	<code>obix:if</code> / <code>obix:else</code>	Conditionals are structural (§5.6)
17	<code>function formatTime(t: number): string</code>	→	<code>derived.formattedTime({ seconds })</code> or a module helper	ES6; destructuring declares dependencies (§10)
18	<code>floorDiv</code> , <code>mod</code> , <code>padLeft</code> builtins	→	<code>Math.floor</code> , <code>%</code> , <code>padStart</code>	Do not reinvent the standard library (§3.6)
19	Keyword table (<code>component</code> , <code>state</code> , <code>action</code> , <code>view</code> , <code>function</code> , <code>choose</code> , <code>typeof</code>)	→	no reserved JavaScript keywords	OBIX does not parse JS expressions (§6.1)
20	<code>.obix.lib</code>	→	plain <code>.js</code> ES module	No bespoke library format (S14)
21	(none — CSS unaddressed)	→	<code><style lang="css" type="scoped"></code>	Presentation is a first-class section (§4)
22	(none)	→	<code>const validation</code> + <code>validity</code> namespace	Form components needed it; FUD's uncertainty half (§11)
23	(none)	→	<code>const effects</code>	Pure actions cannot make a clock tick (§6.3)
24	(none)	→	canonical DOP IR + <code>Timer.dop.json</code>	One definition, four adapters, portable declarations (§7)

#	Draft 0.1	→	Draft 0.2	Reason
25	<code>test Timer "desc" { given / when / expect }</code>	→	preserved , inside <code>suite Timer { ... }</code>	Right idiom; only the wrapper is new (§22)
26	<code>layer state render integration</code>	→	implied by assertion subject and artifact	The concept survives; the ceremony does not (X5)
27	<code>expect { property == value }</code>	→	<code>expect state.property == value</code>	Explicit subject; <code>derived</code> , <code>validity</code> , <code>render</code> are also subjects (§22.3)
28	<code>unchanged</code> keyword	→	<code>expect state unchanged</code> / <code>post state unchanged</code>	Preserved verbatim (I14)
29	<code>pre</code> , <code>post</code> , <code>old</code> keywords	→	<code>contract Tick { pre ... post ... old state.x }</code>	Preserved; given a home in the contract artifact (§23.2)
30	<code>invariant</code> keyword	→	<code>invariant Name { for any state: ... }</code>	Preserved (§23.2)
31	<code>role</code> , <code>label</code> , <code>keyboard</code> , <code>focusable</code> , <code>aria</code> keywords	→	<code>element Name { select role ... expect keyboard ... }</code>	Preserved; selection is by accessibility tree (§23.3)
32	(none)	→	<code>equivalence Name { ... across data, functional, oop, reactive }</code>	The invariant that licenses one component definition (§20)
33	<code>Component.test.obix</code> = behavioural	→	unchanged	Inherited (I2)
34	<code>Component.obix.test</code> = contract	→	unchanged , extended with equivalence	Inherited (I3)
35	Contract runs before behavioural	→	unchanged	Inherited (I5)
36	<code>ctx.state.clicked = !ctx.state.clicked</code>	→	<code>Toggle(state) { return { ...state, clicked: !state.clicked }; }</code>	Four architectural features require the previous state to survive (§3.7)
37	<code>new DOPAdapter(logic).toFunctional()</code>	→	<code>functional(Timer)</code> — or <code>fromLegacy(logic)</code> during migration	Adapters consume the IR (§16–19, §29)
38	functional / OOP duality	→	Data Functional OOP Reactive over one definition	Generalises the prototype's proven bijection (§20, O2)
39	<code>appsuite</code> , <code>app</code> , <code>domain</code> , <code>host</code>	→	reserved, undesigned	OBIXverse waits for a stable component syntax (Q12)

Appendix B — Diagnostic code families

Prefix	Domain
<code>OBIX-F###</code>	File, naming, section structure
<code>OBIX-S###</code>	Style section, scoping
<code>OBIX-T###</code>	Template, bindings, directives, events
<code>OBIX-J###</code>	Script, declarations, purity, serializability
<code>OBIX-A###</code>	Accessibility
<code>OBIX-C###</code>	Contract and surface drift
<code>OBIX-E###</code>	Adapter equivalence failure

Prefix

Domain

OBIX-L###

Legacy compatibility and deprecation

OBIX — Obi, the heart. One component definition. Four ways to hold it. One invariant that keeps them honest.

OBINexus Computing — Nnamdi Michael Okpala — Draft 0.2 — 28 August 2026